

Data Structures for *Map ADT*

	Find	Insert	Delete
UnsortedArrayMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
UnsortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
SortedArrayMap	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
SortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
BinarySearchTreeMap			

Data Structures for *Map ADT*

	Find	Insert	Delete
UnsortedArrayMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
UnsortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
SortedArrayMap	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
SortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
BinarySearchTreeMap	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$

Data Structures for *Map ADT*

	Find	Insert	Delete
UnsortedArrayMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
UnsortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
SortedArrayMap	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
SortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
BinarySearchTreeMap	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$
AVLTreeMap			

Data Structures for *Map ADT*

	Find	Insert	Delete
UnsortedArrayMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
UnsortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
SortedArrayMap	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
SortedLinkedListMap	$\theta(n)$	$\theta(n)$	$\theta(n)$
BinarySearchTreeMap	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$	$\theta(n)$ Also, $\theta(h)$
AVLTreeMap	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$

AVL Trees

AVL Trees

AVL Trees

Height-Balance Property

AVL Trees

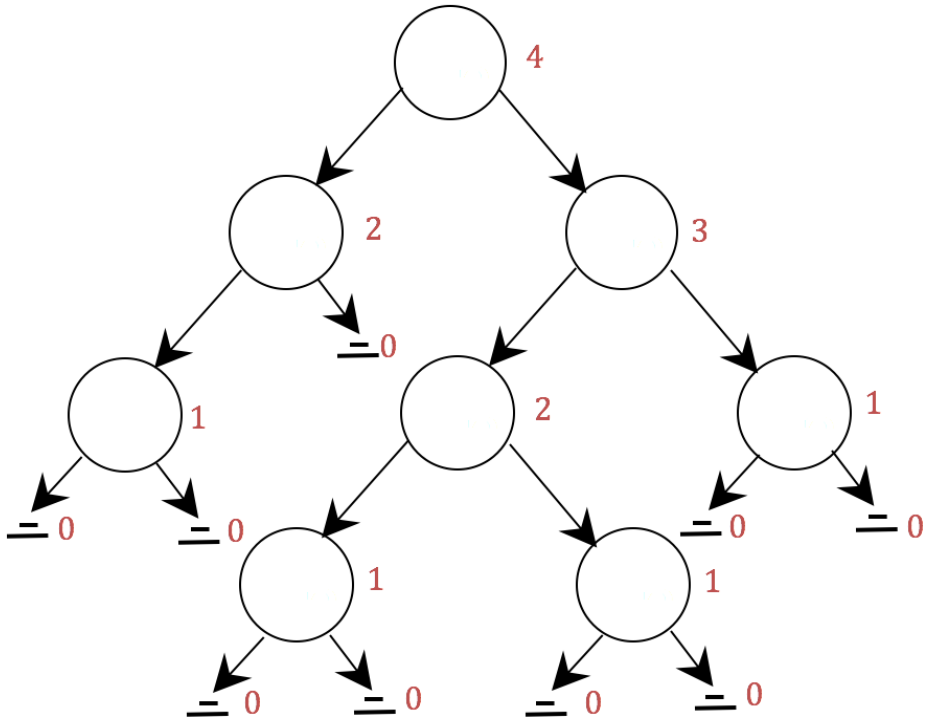
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Trees

Height-Balance Property

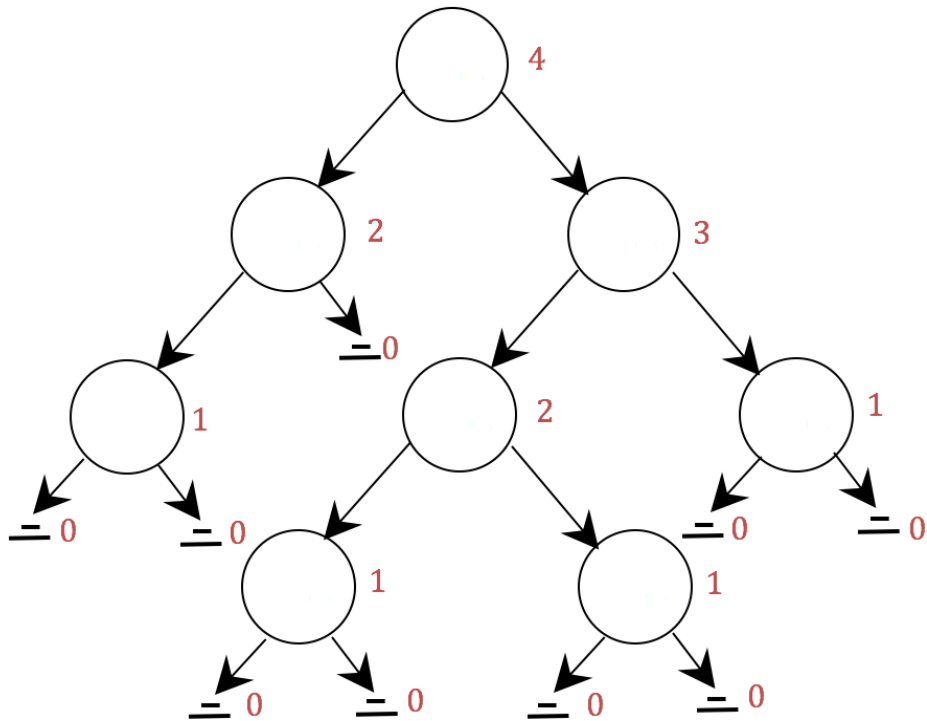
For every node n of T , the heights of the children of n differ by at most 1.



AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

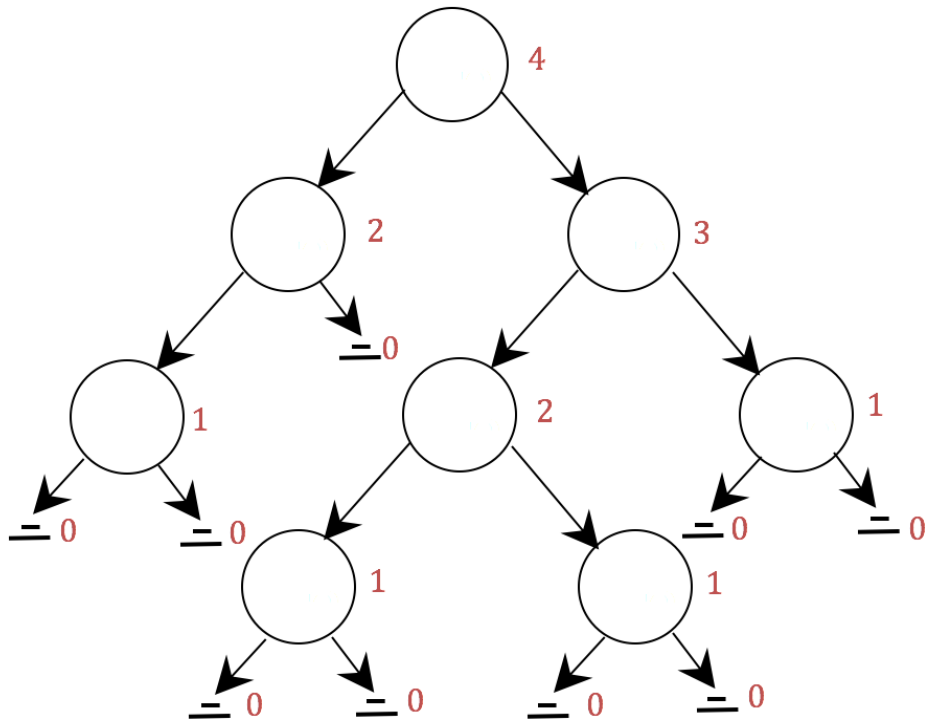


Height Balanced

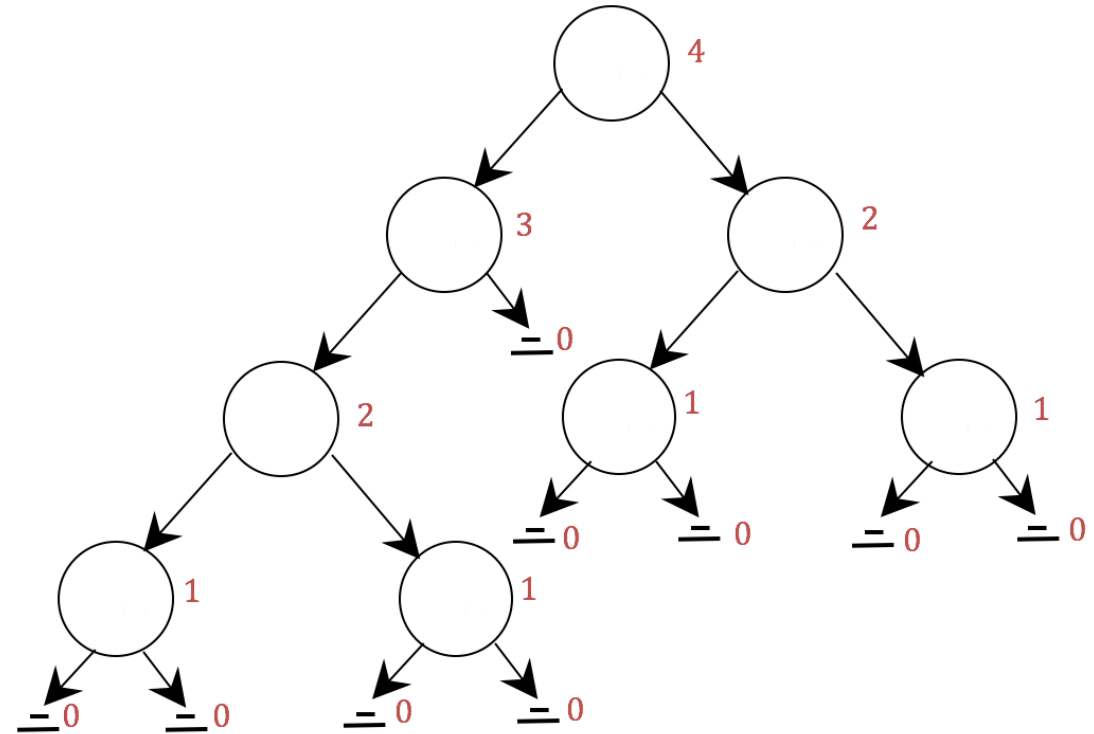
AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.



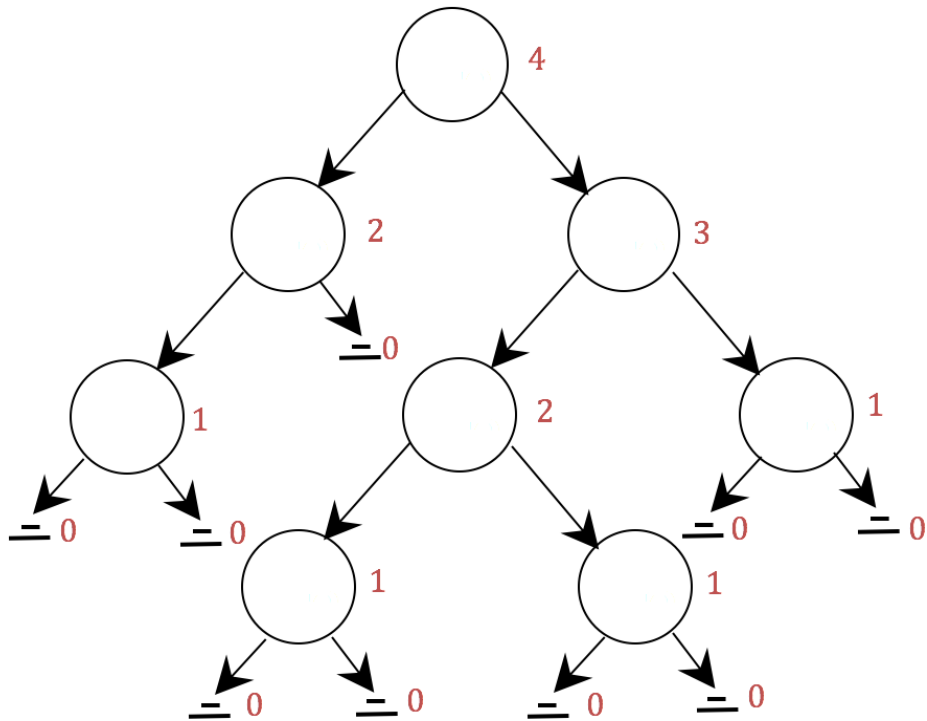
Height Balanced



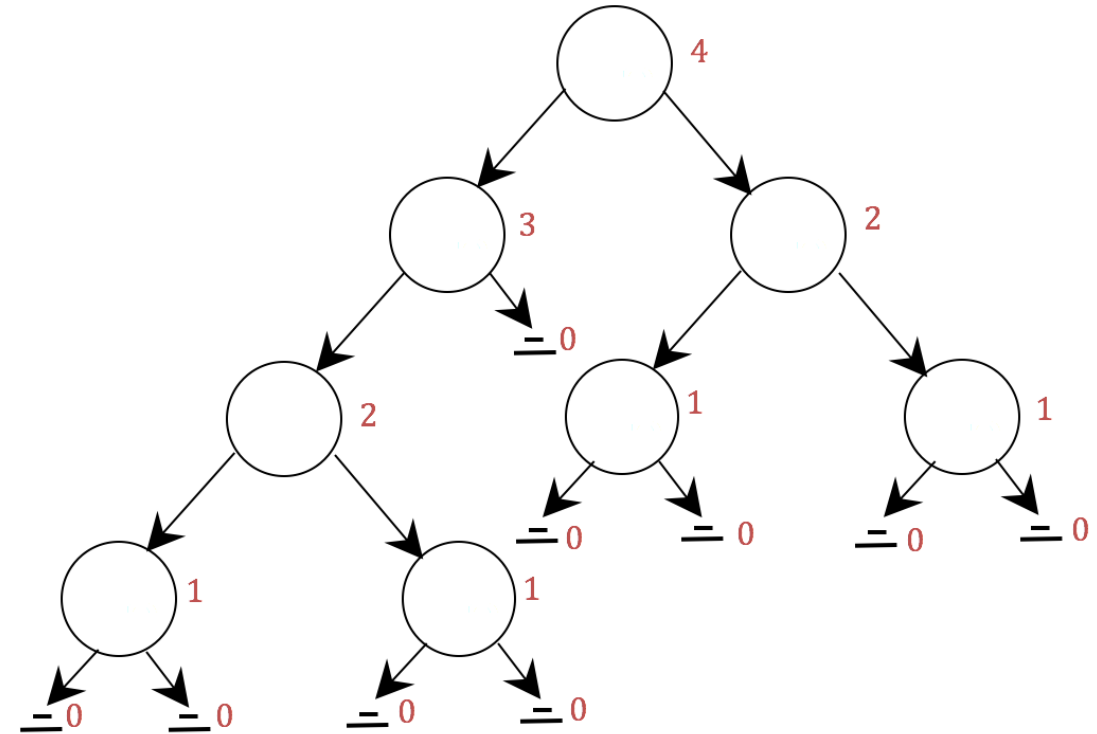
AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.



Height Balanced



Not Height Balanced

AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree.

AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

- 1.
- 2.

AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
- 2.

AVL Trees

Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**

AVL Trees

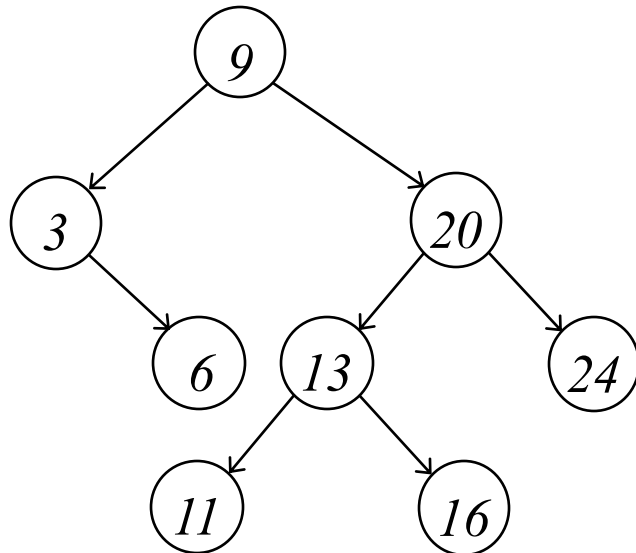
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**



AVL Trees

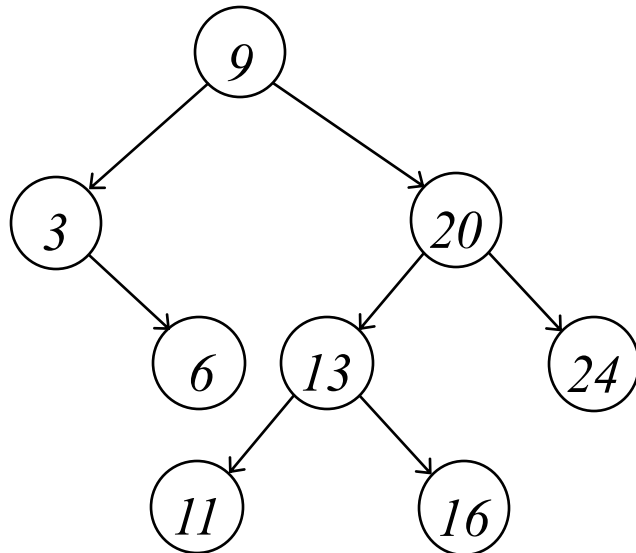
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**



AVL Tree

AVL Trees

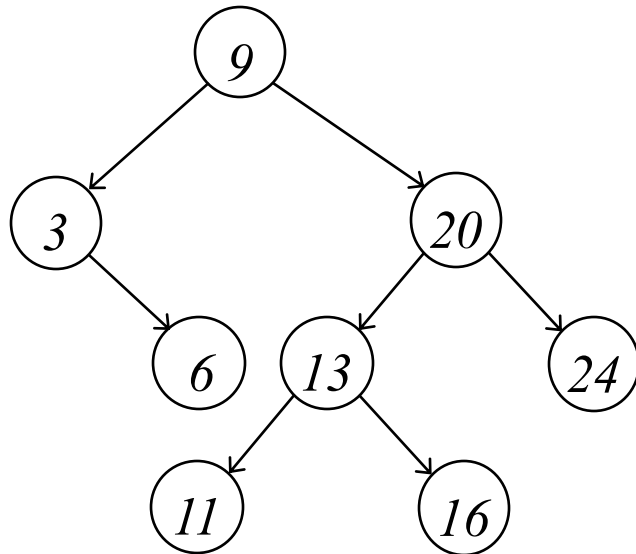
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

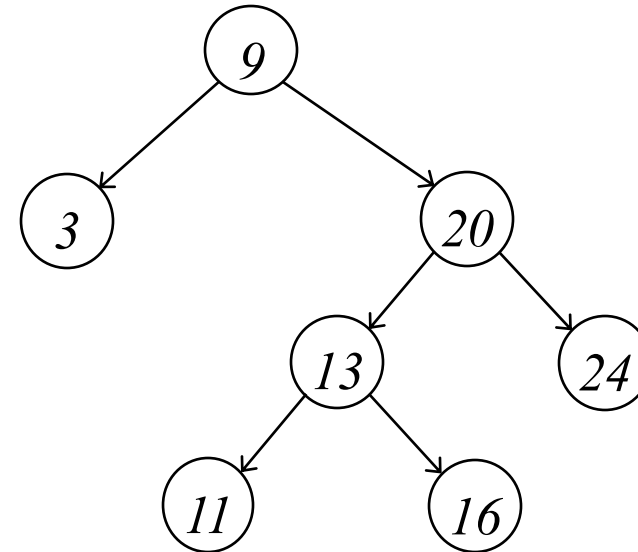
AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**



AVL Tree



AVL Trees

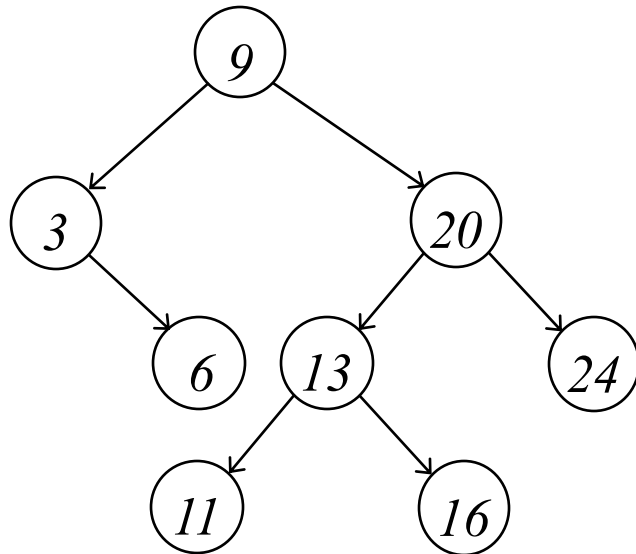
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

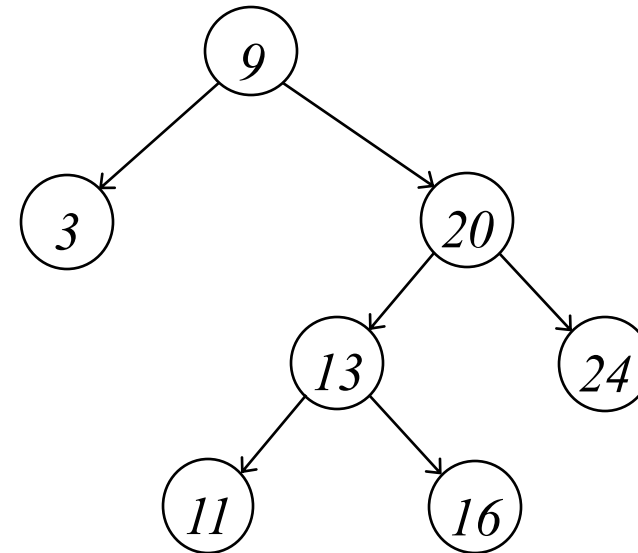
AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**



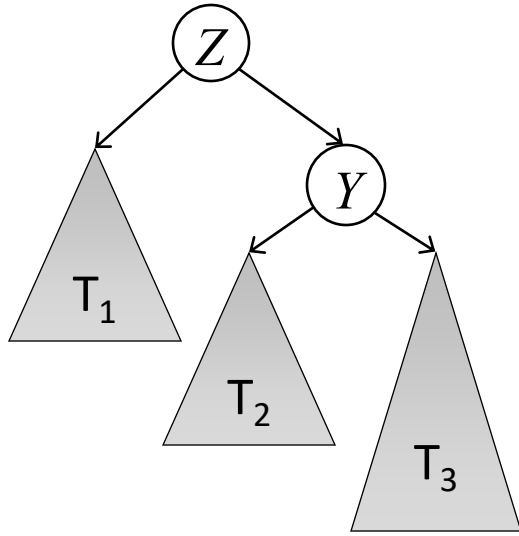
AVL Tree



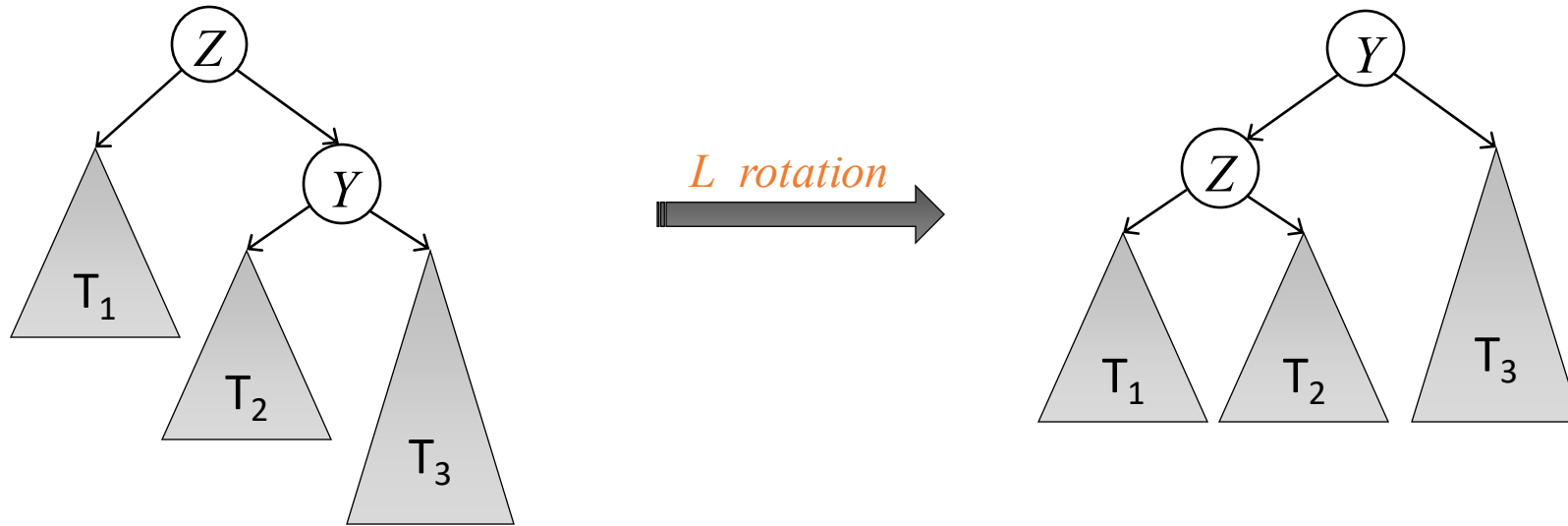
Not an AVL Tree

Single Rotations

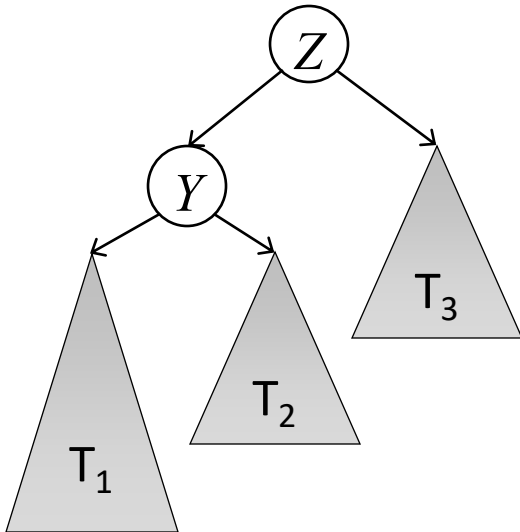
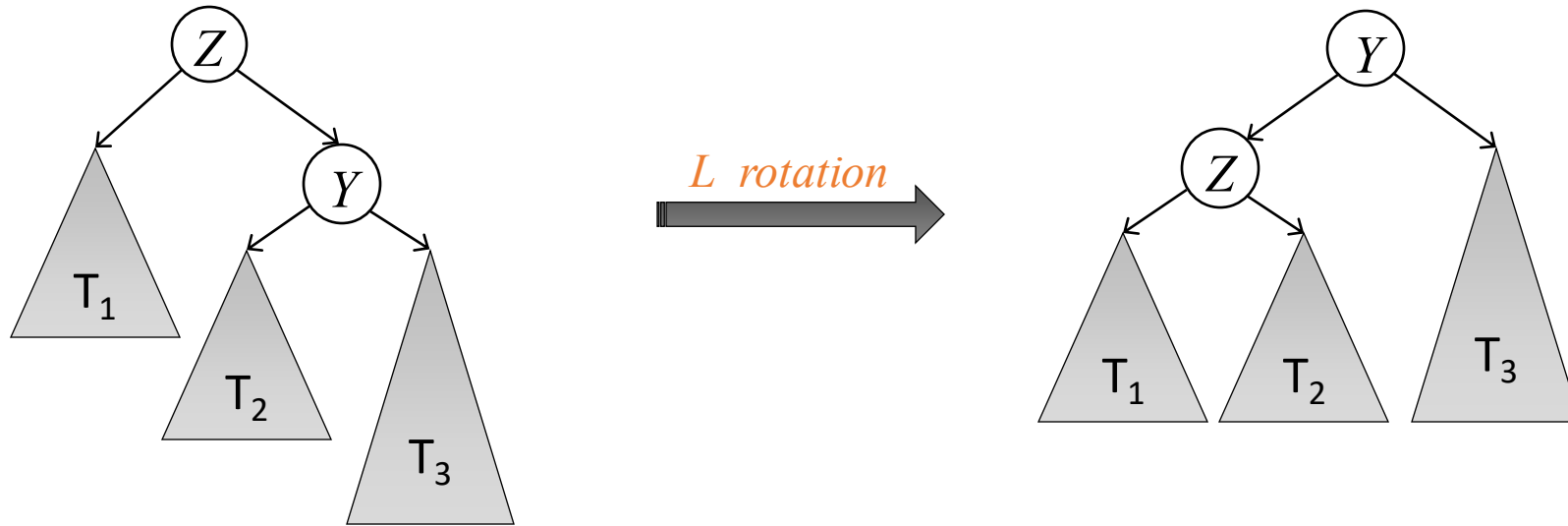
Single Rotations



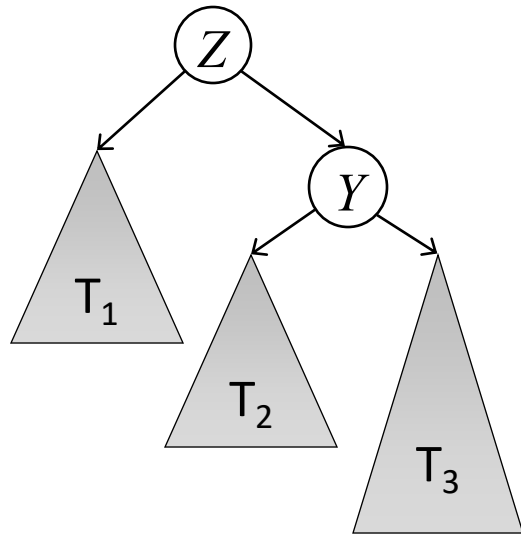
Single Rotations



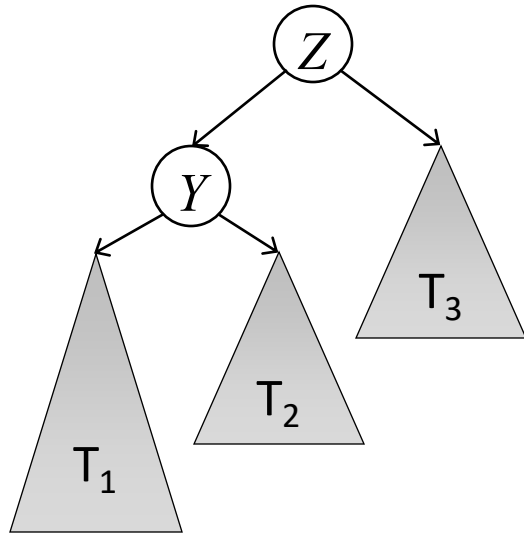
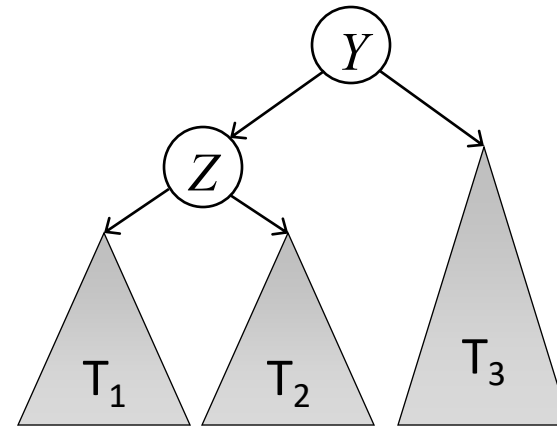
Single Rotations



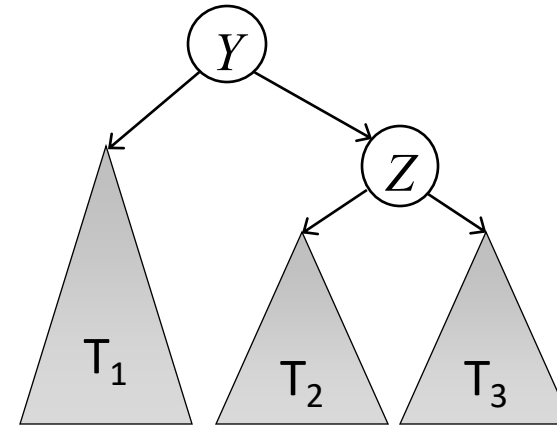
Single Rotations

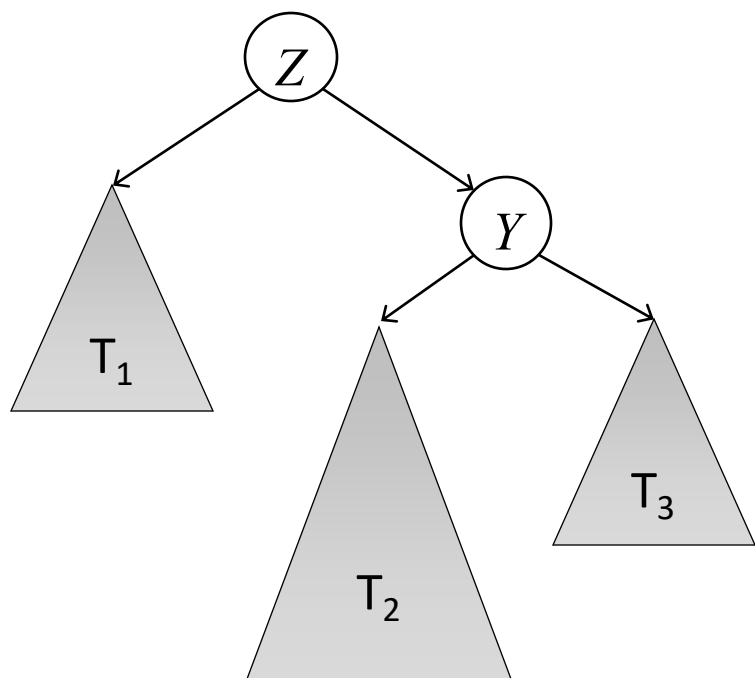


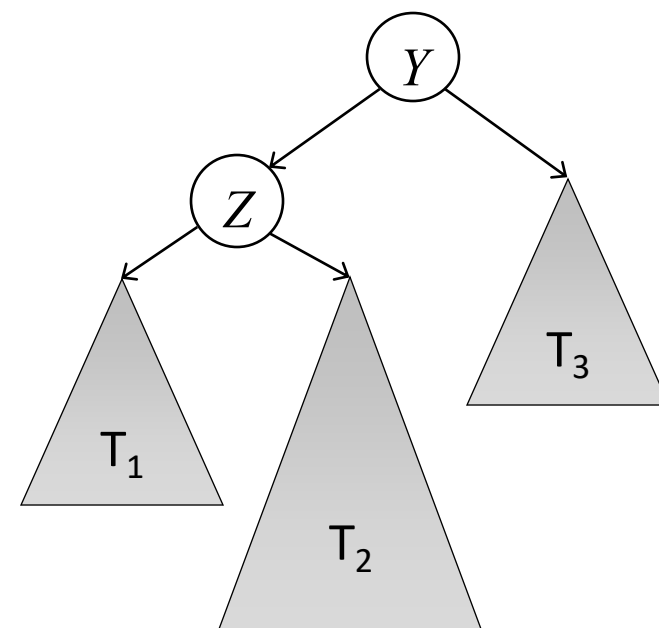
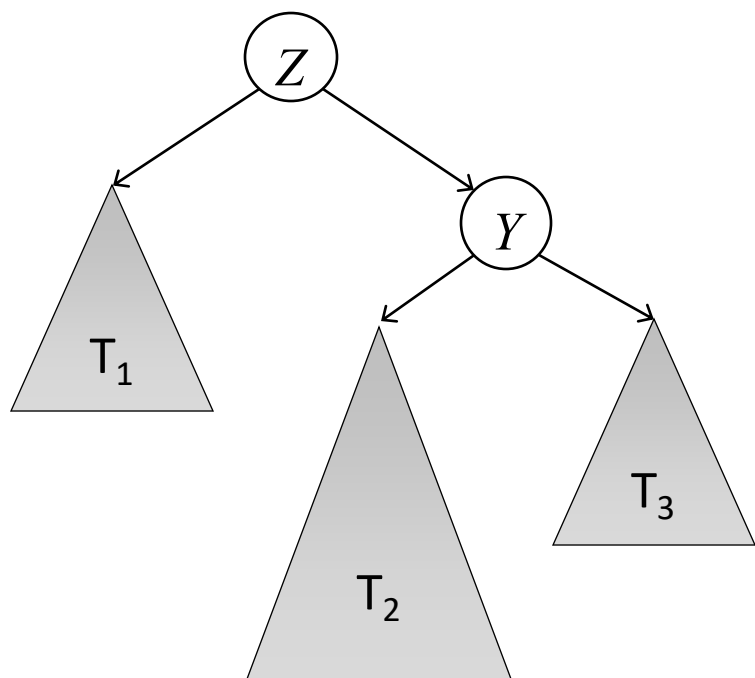
L rotation

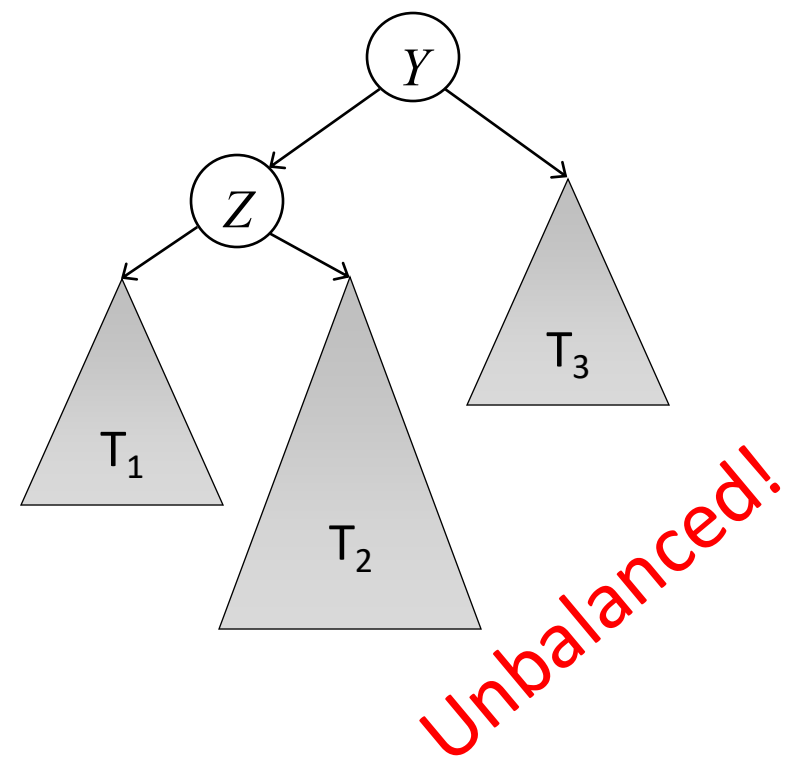
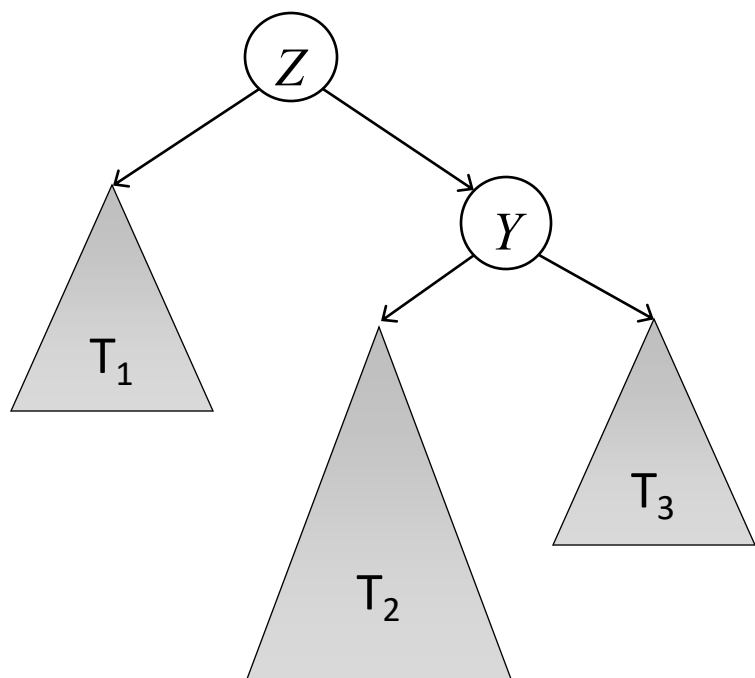


R rotation

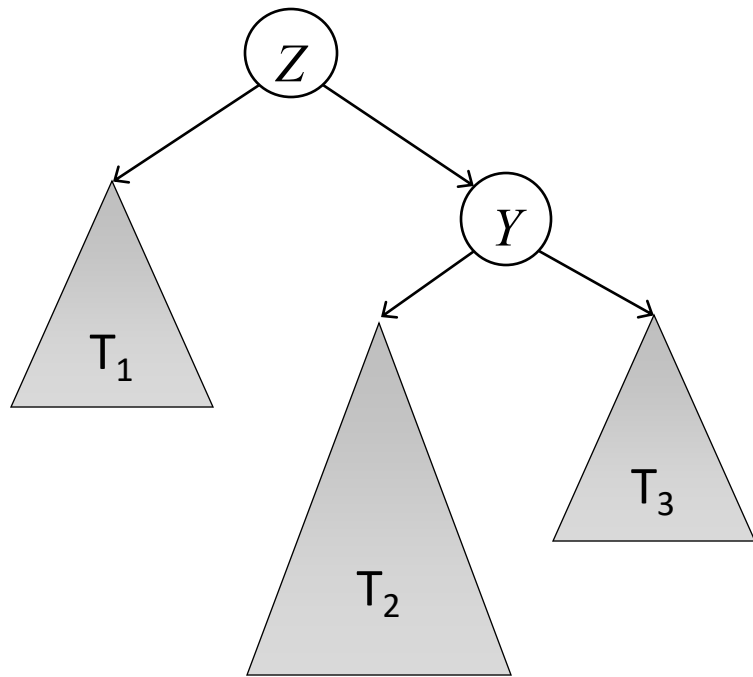




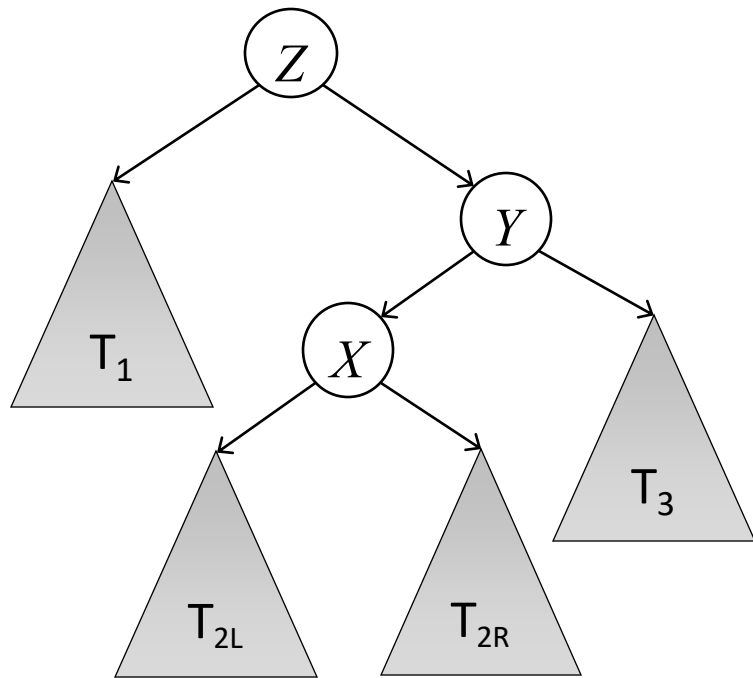




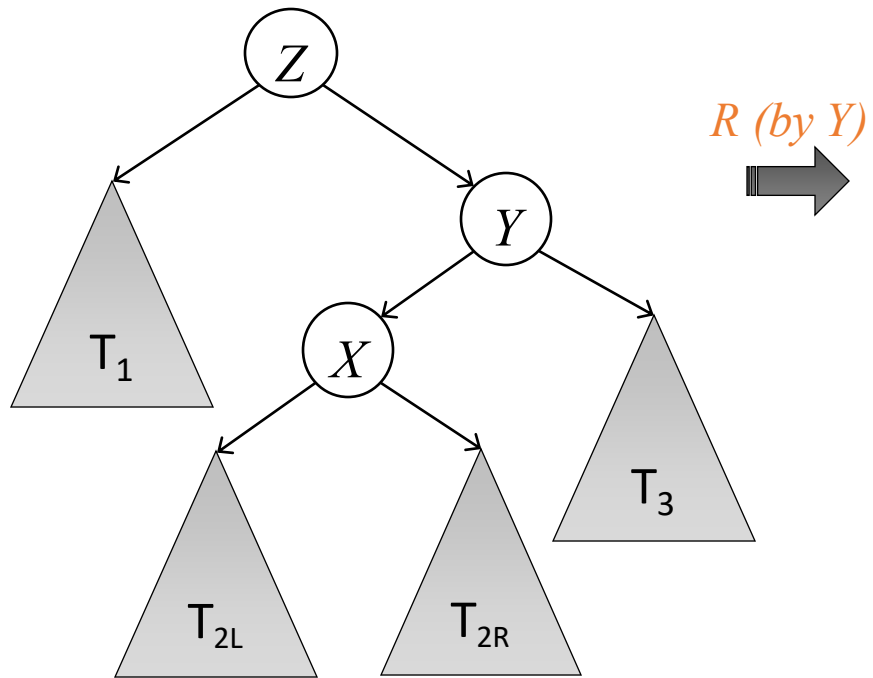
Double Rotations



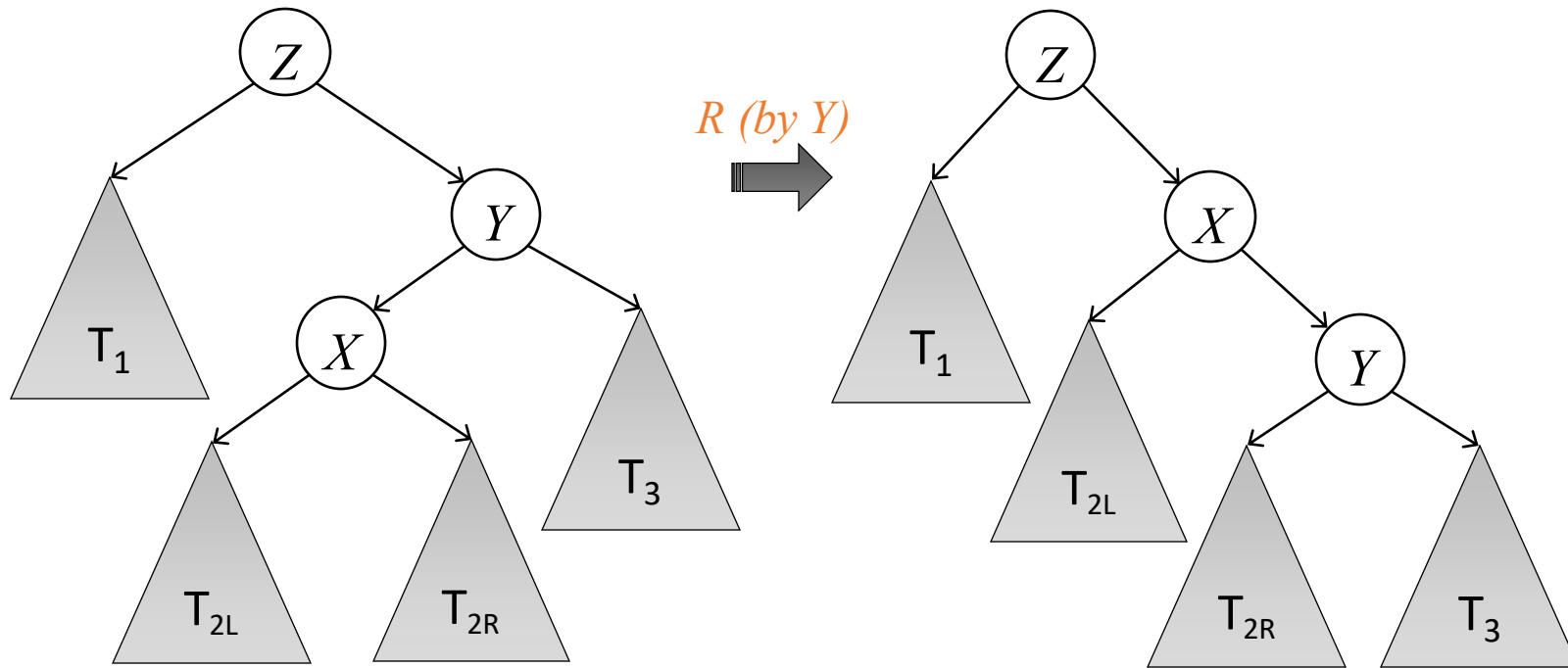
Double Rotations



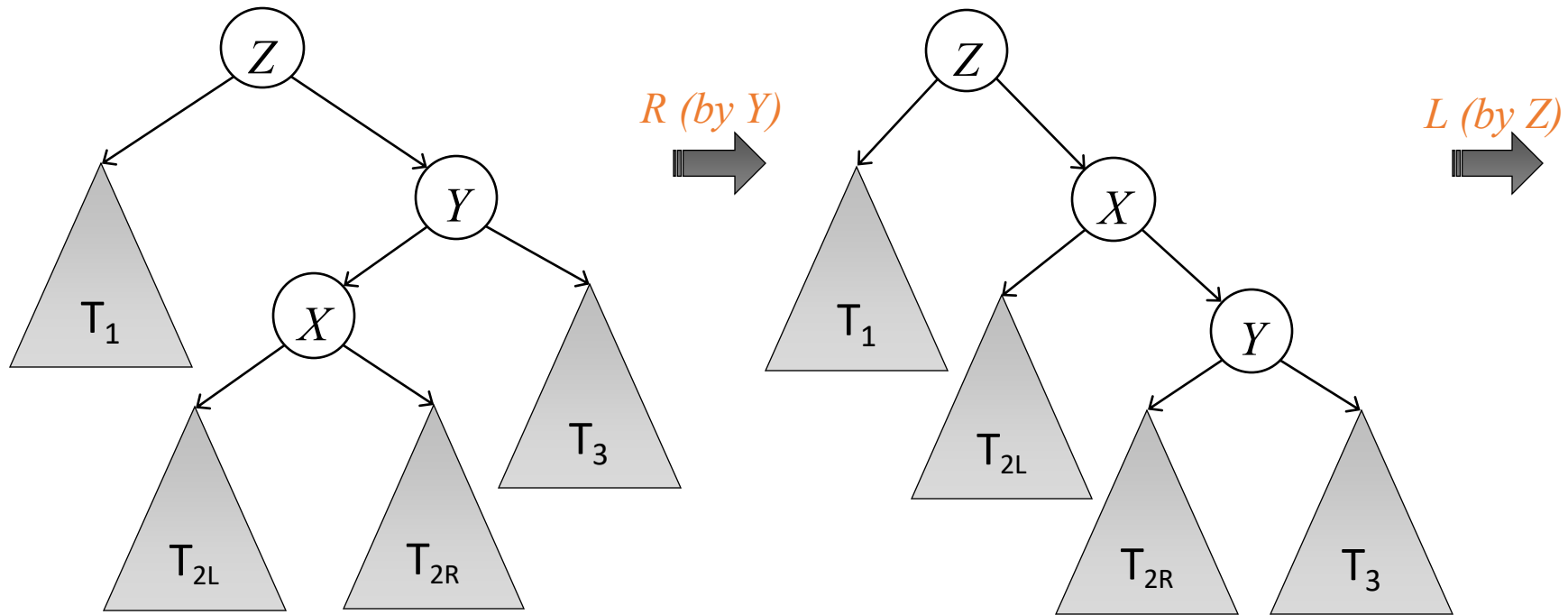
Double Rotations



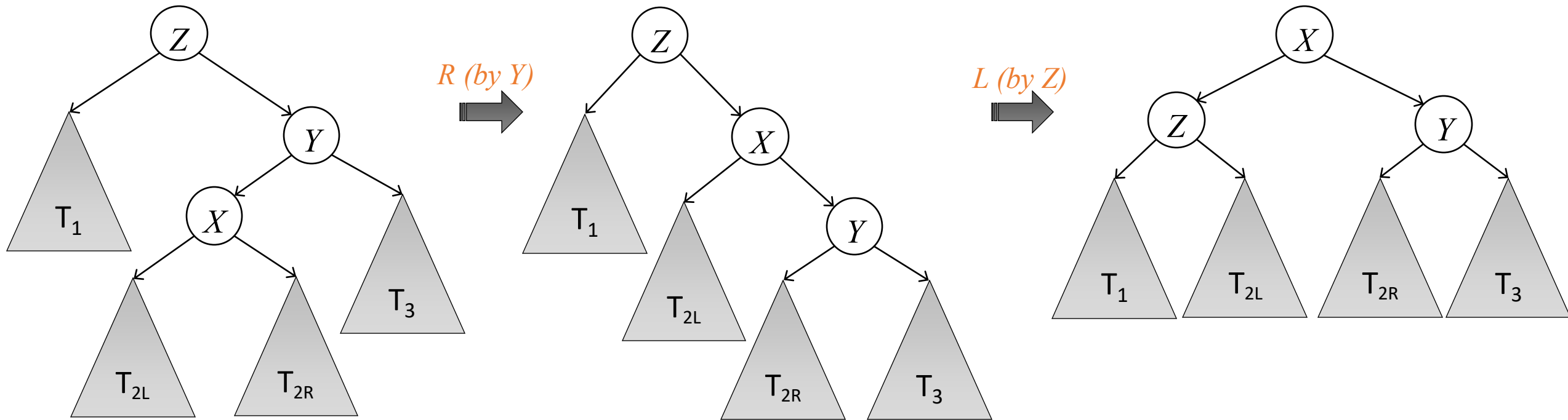
Double Rotations



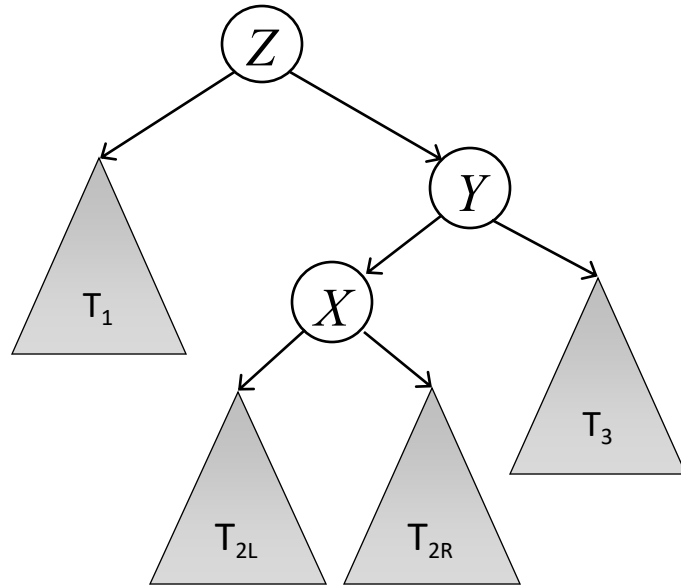
Double Rotations



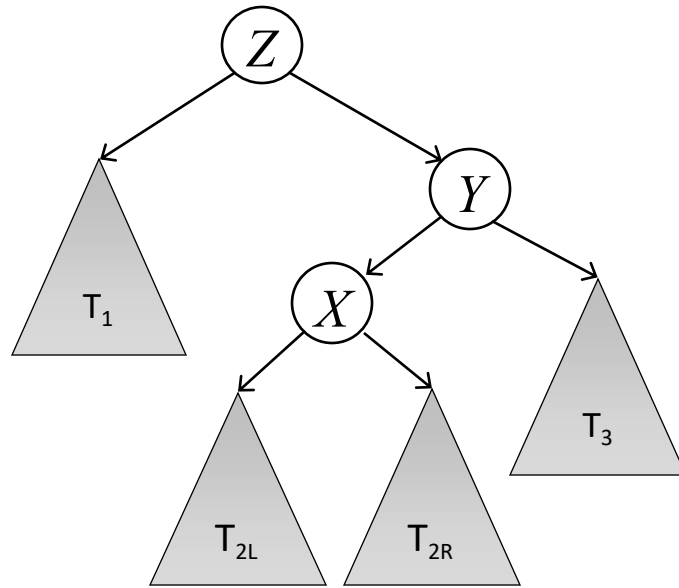
Double Rotations



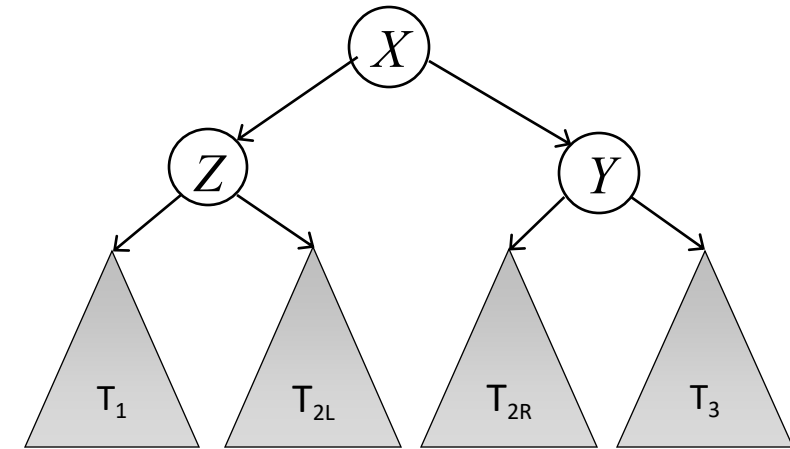
Double Rotations



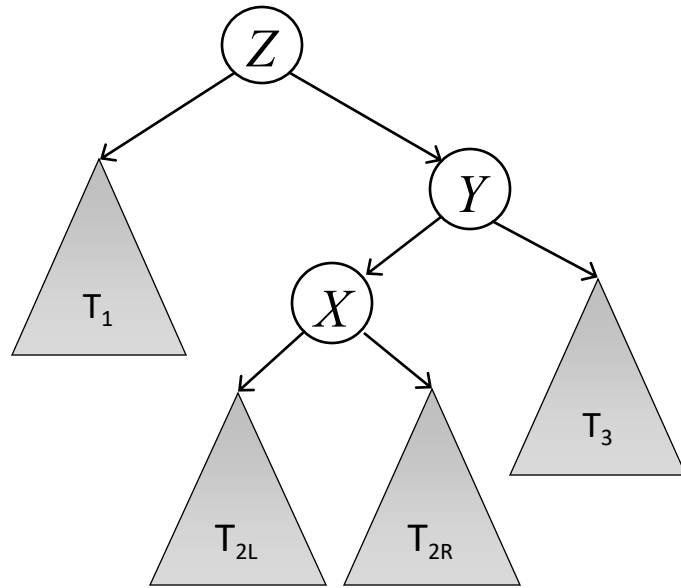
Double Rotations



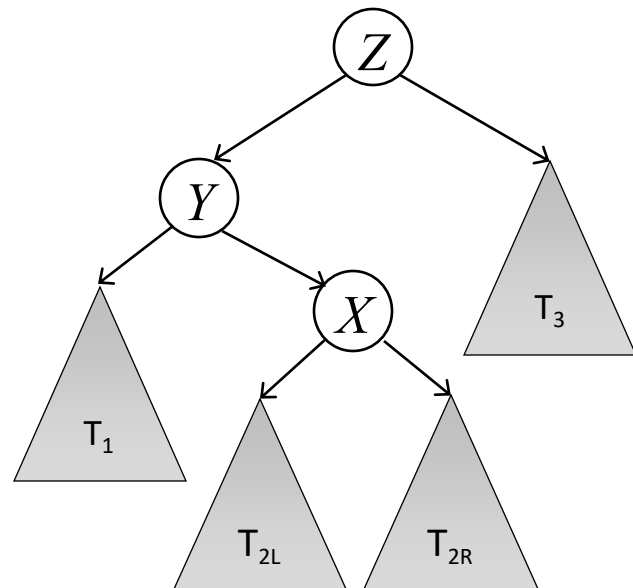
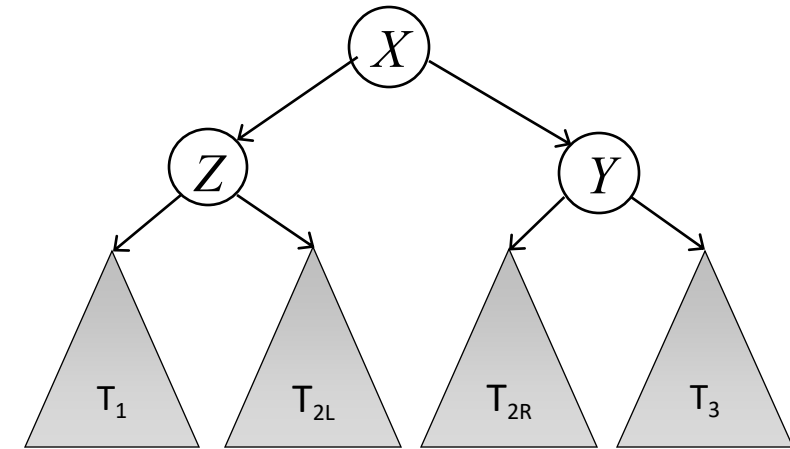
RL rotation →



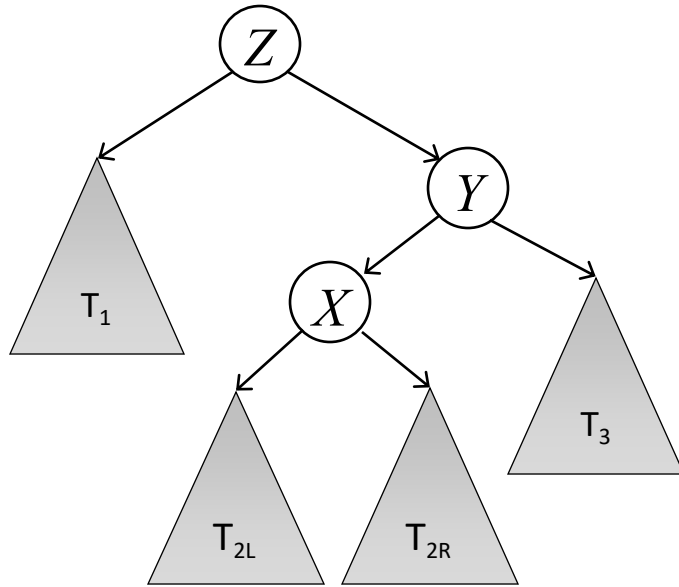
Double Rotations



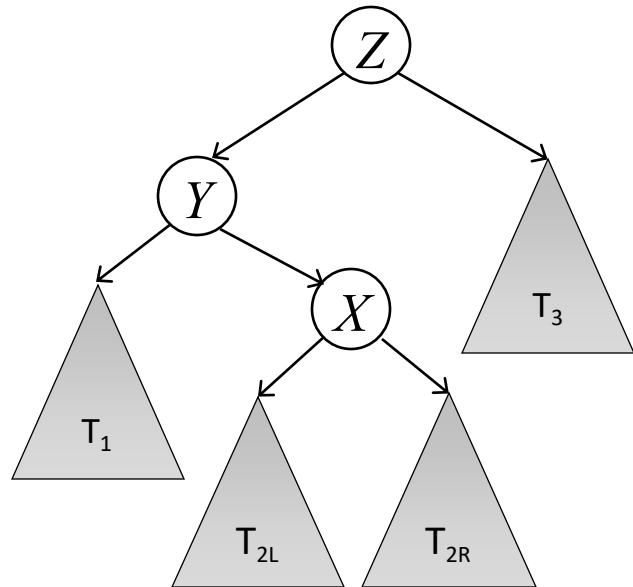
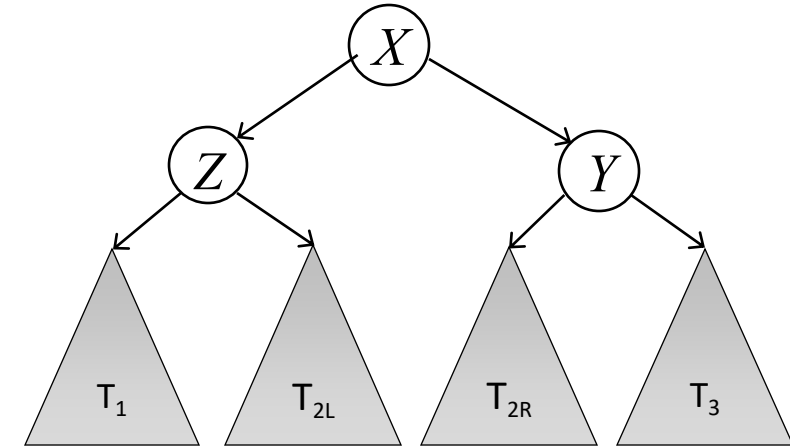
RL rotation →



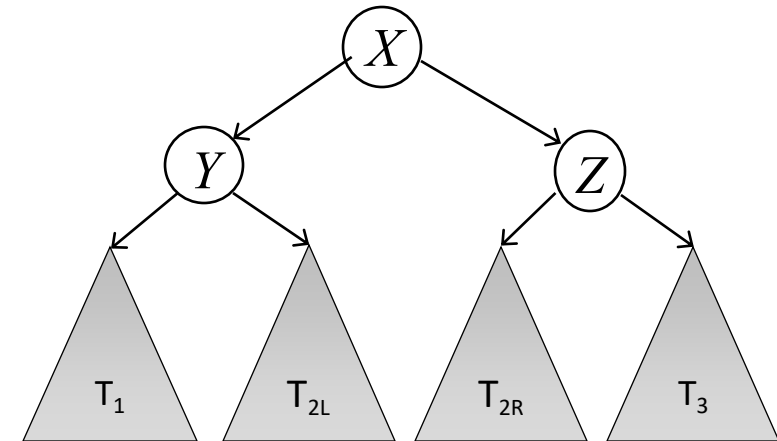
Double Rotations



RL rotation →



LR rotation →



AVL Trees

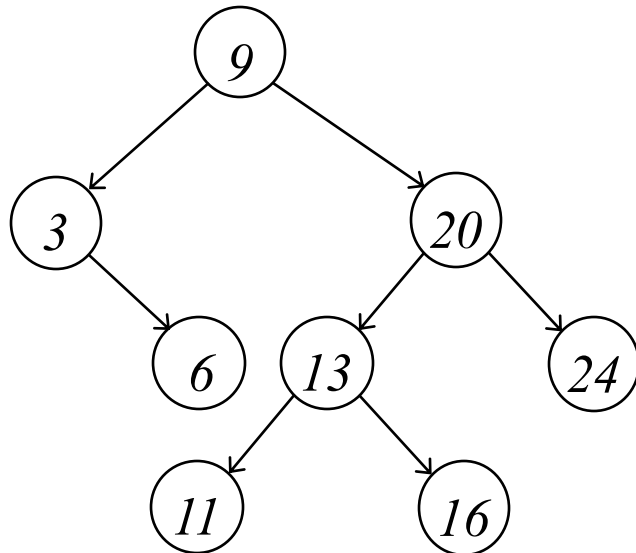
Height-Balance Property

For every node n of T , the heights of the children of n differ by at most 1.

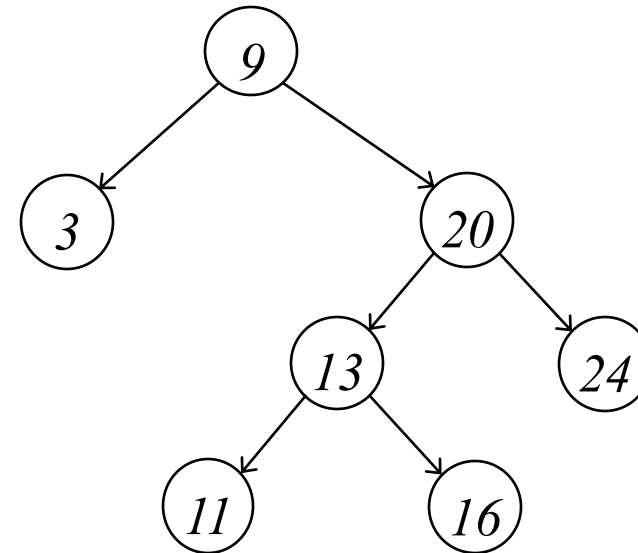
AVL Tree

Let T be a binary tree. We say that T is an *AVL Tree* if:

1. It is a **binary search tree**
2. It satisfies the **height-balance property**



AVL Tree



Not an AVL Tree

AVL Insert

AVL Insert

Insert k into an *AVL tree*

AVL Insert

Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*.

AVL Insert

Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*.
As a result, the balancing property could be violated

AVL Insert

Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*.
As a result, the balancing property could be violated
- Walk from the new node (with k) upwards on the insertion path.

AVL Insert

Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*. As a result, the balancing property could be violated
- Walk from the new node (with k) upwards on the insertion path.
- Let z be the first node in this walk that is not balanced (The difference between z 's two children is 2)

AVL Insert

Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*. As a result, the balancing property could be violated
- Walk from the new node (with k) upwards on the insertion path.
- Let z be the first node in this walk that is not balanced (The difference between z 's two children is 2)
- Let y be z 's higher child (k was inserted into y 's subtree)

AVL Insert

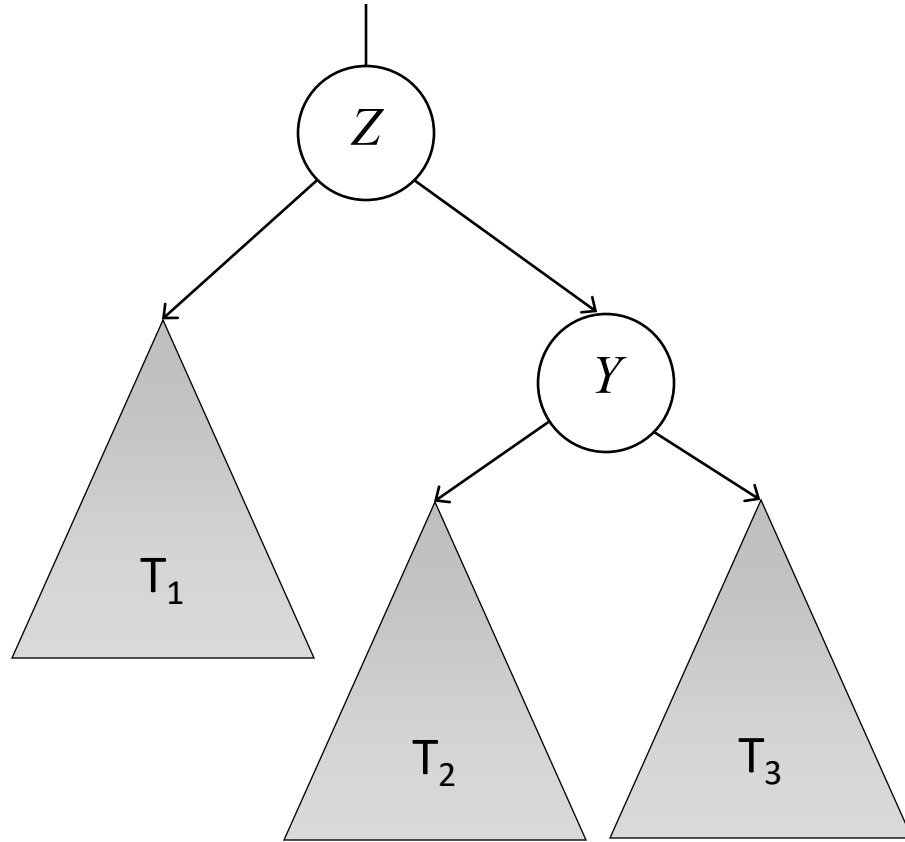
Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*. As a result, the balancing property could be violated
- Walk from the new node (with k) upwards on the insertion path.
- Let z be the first node in this walk that is not balanced (The difference between z 's two children is 2)
- Let y be z 's higher child (k was inserted into y 's subtree)
- Determine if you are in the outside-case or in the inside-case
 - If you are in the outside-case do the corresponding single rotation
 - If you are in the inside-case do the corresponding double rotation

AVL Insert – Outside Case

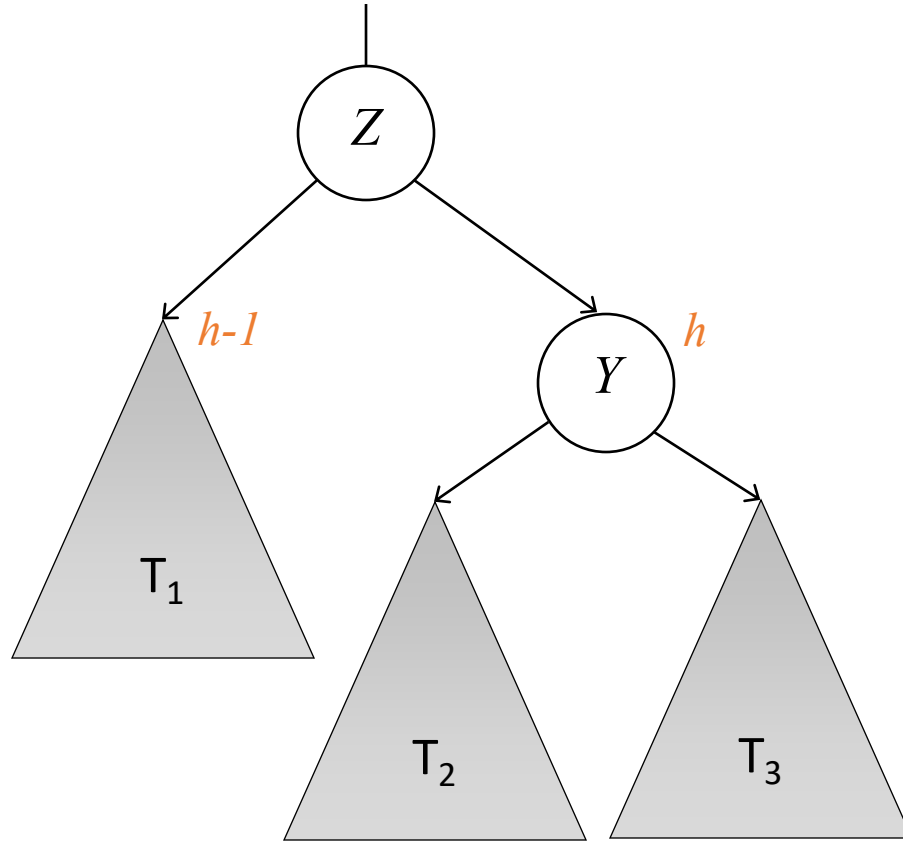
AVL Insert – Outside Case

Before the Insert operation



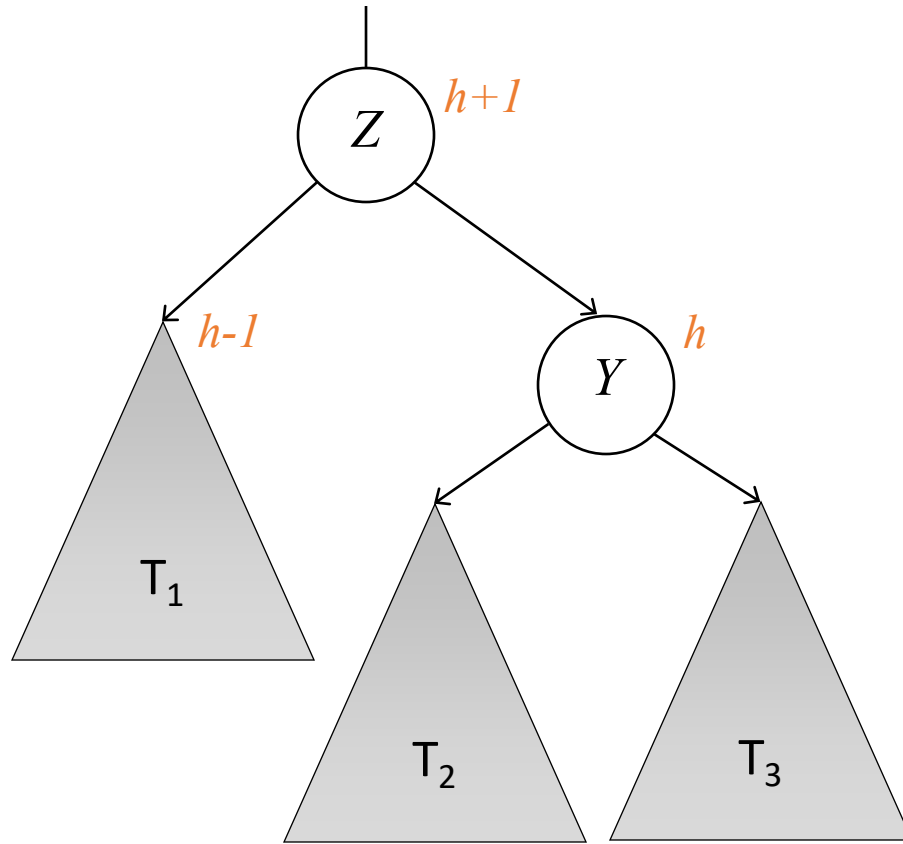
AVL Insert – Outside Case

Before the Insert operation



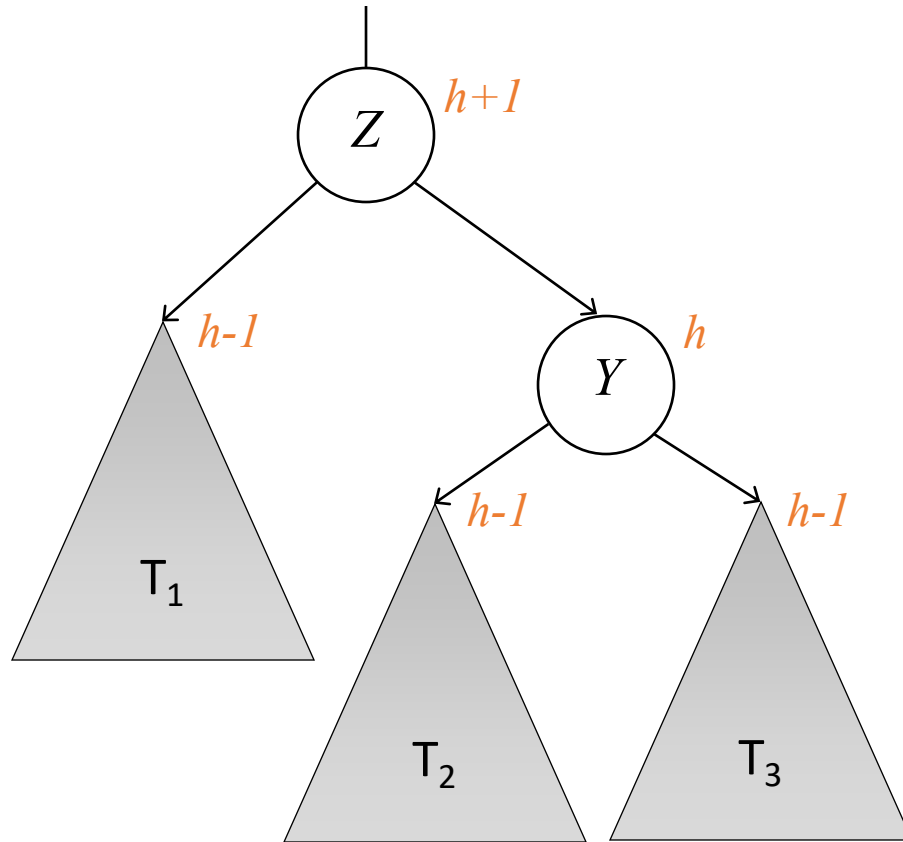
AVL Insert – Outside Case

Before the Insert operation



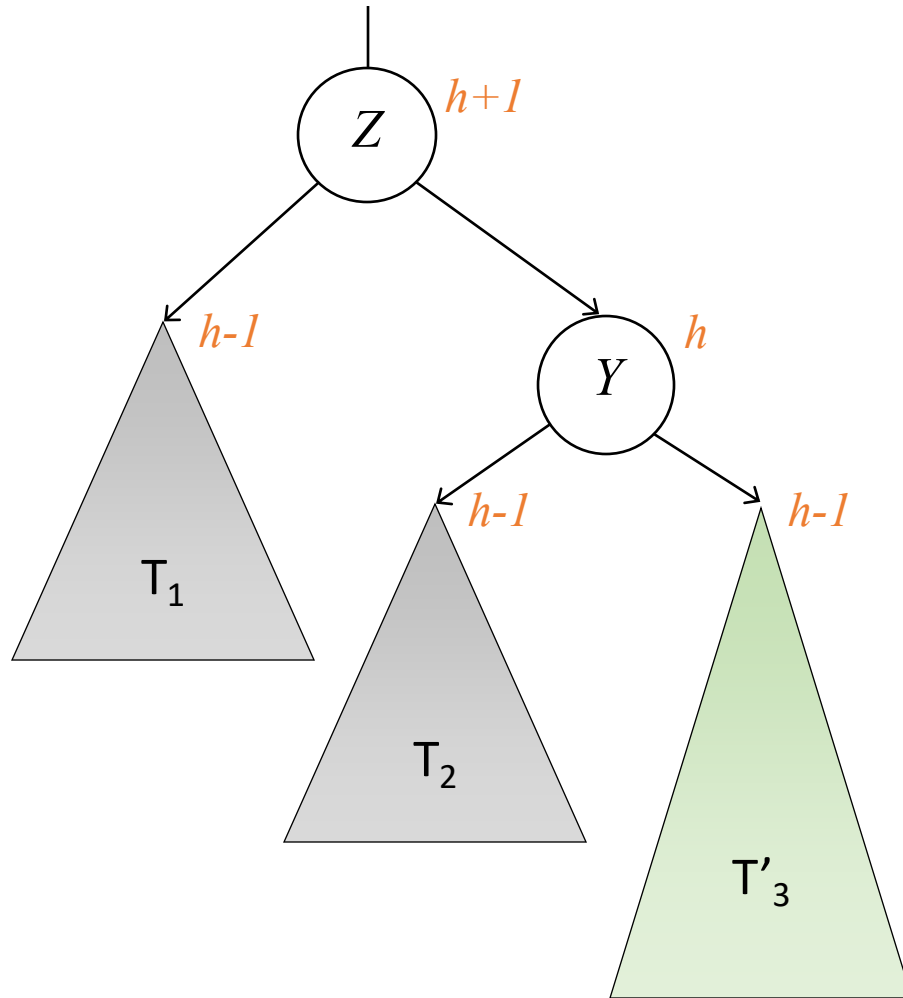
AVL Insert – Outside Case

Before the Insert operation



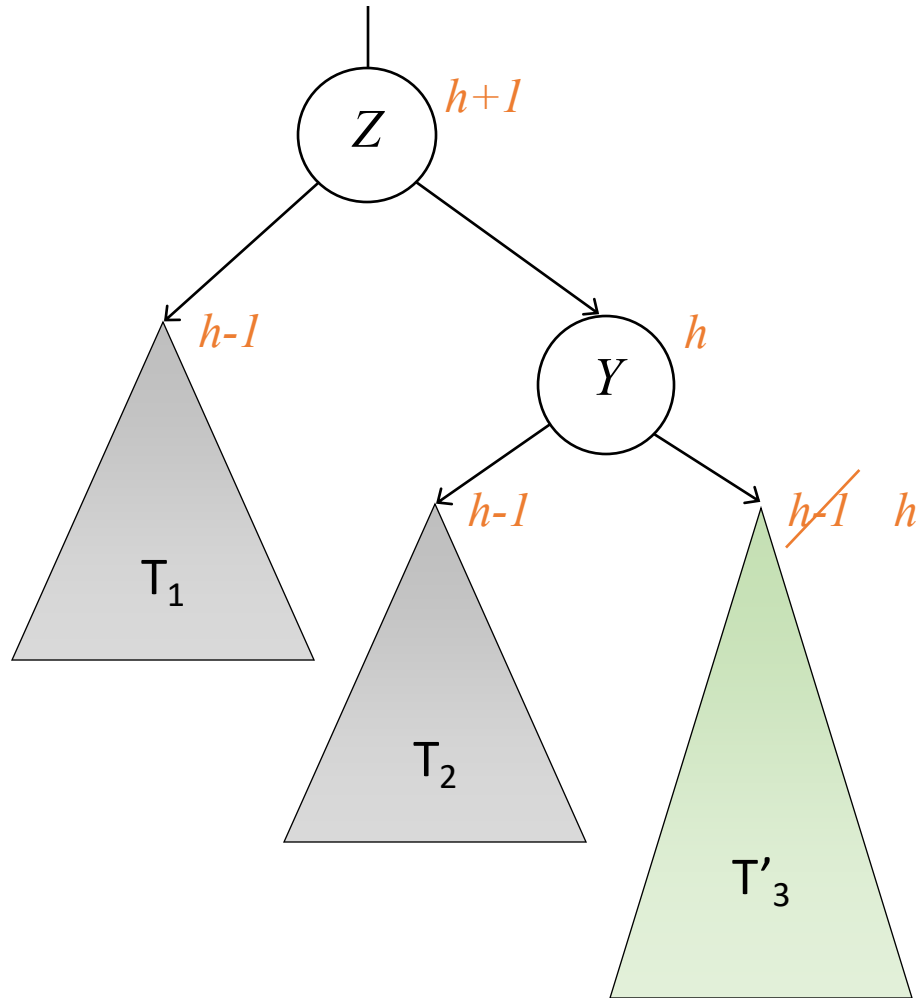
AVL Insert – Outside Case

After the Insert operation



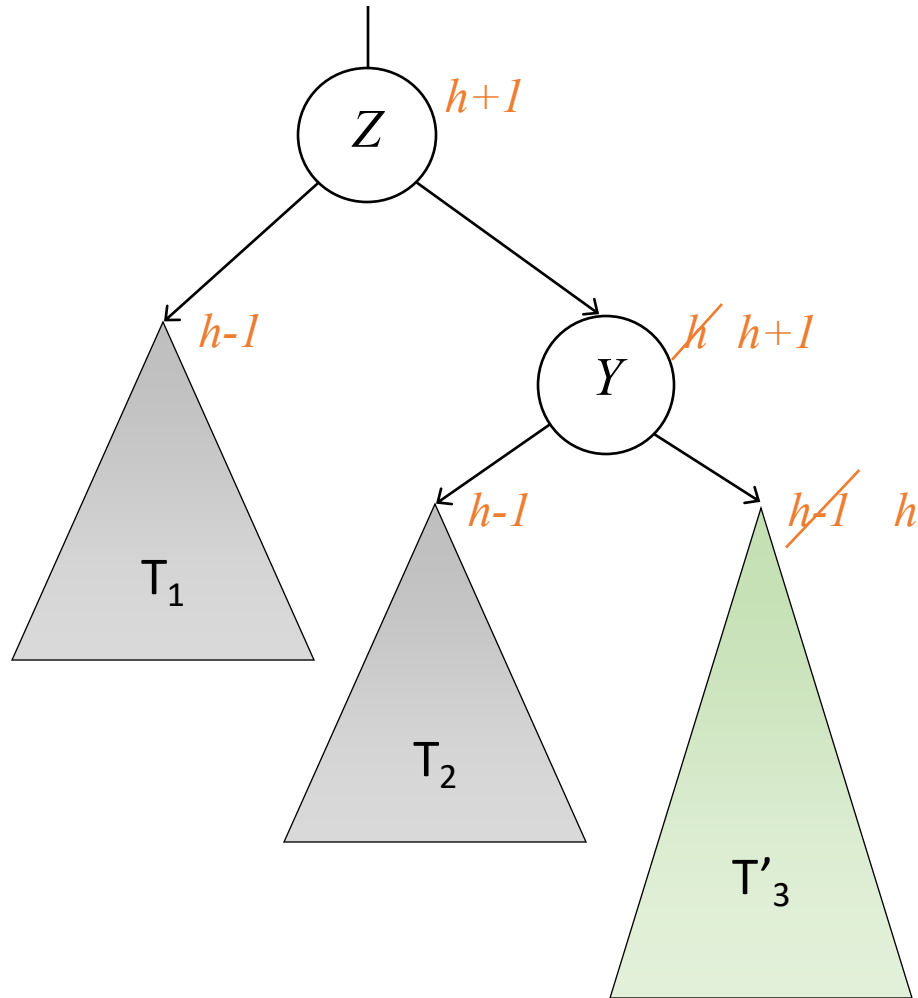
AVL Insert – Outside Case

After the Insert operation



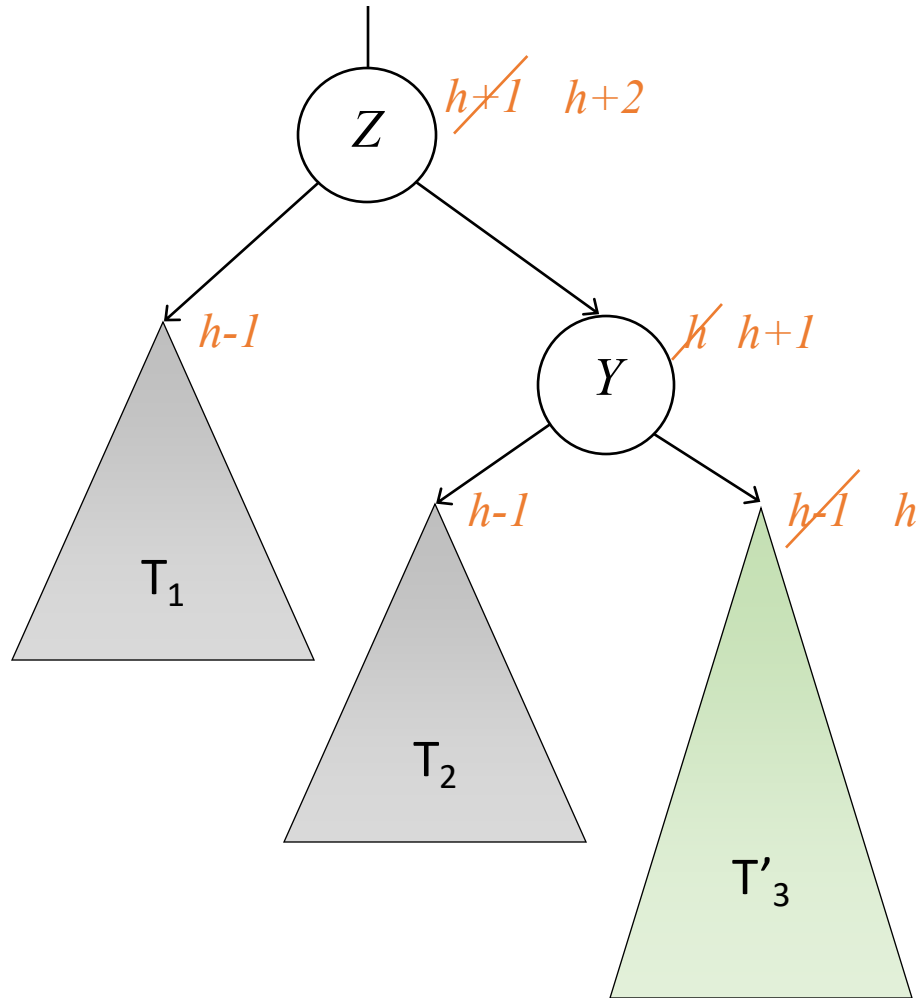
AVL Insert – Outside Case

After the Insert operation



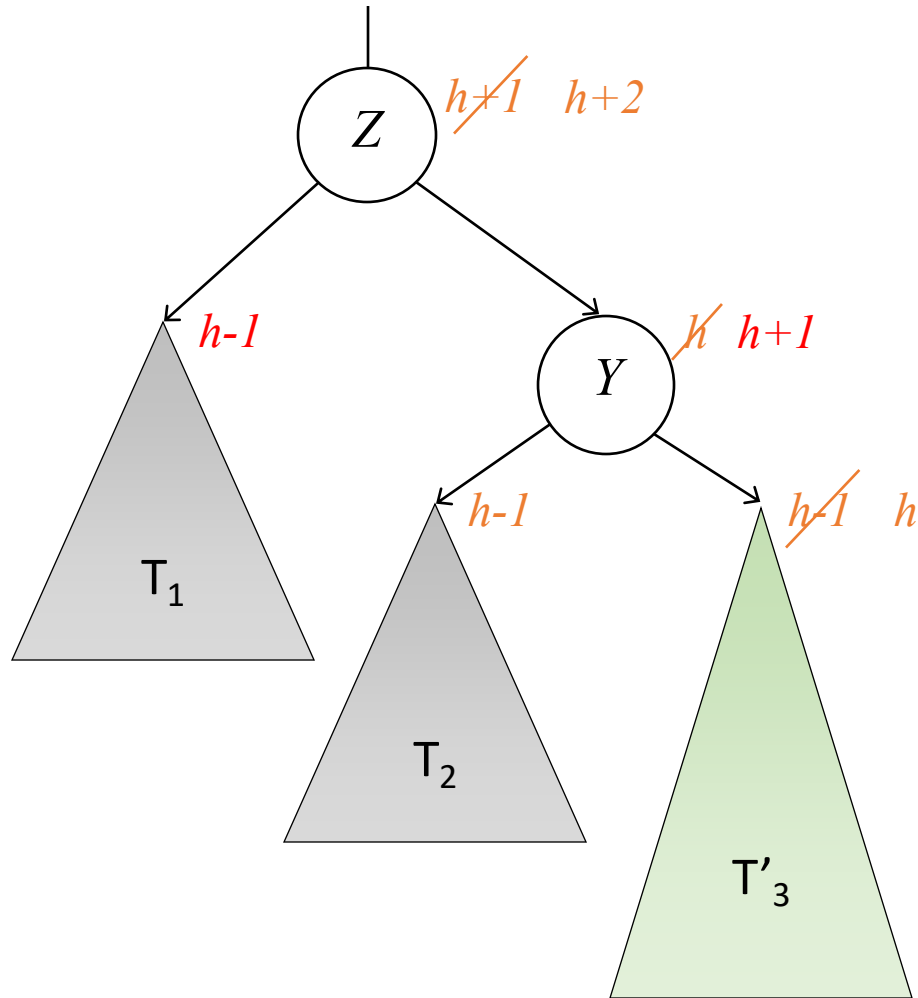
AVL Insert – Outside Case

After the Insert operation



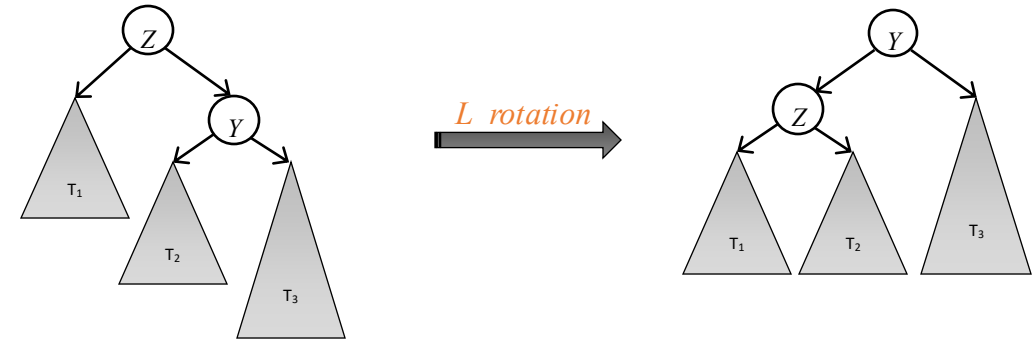
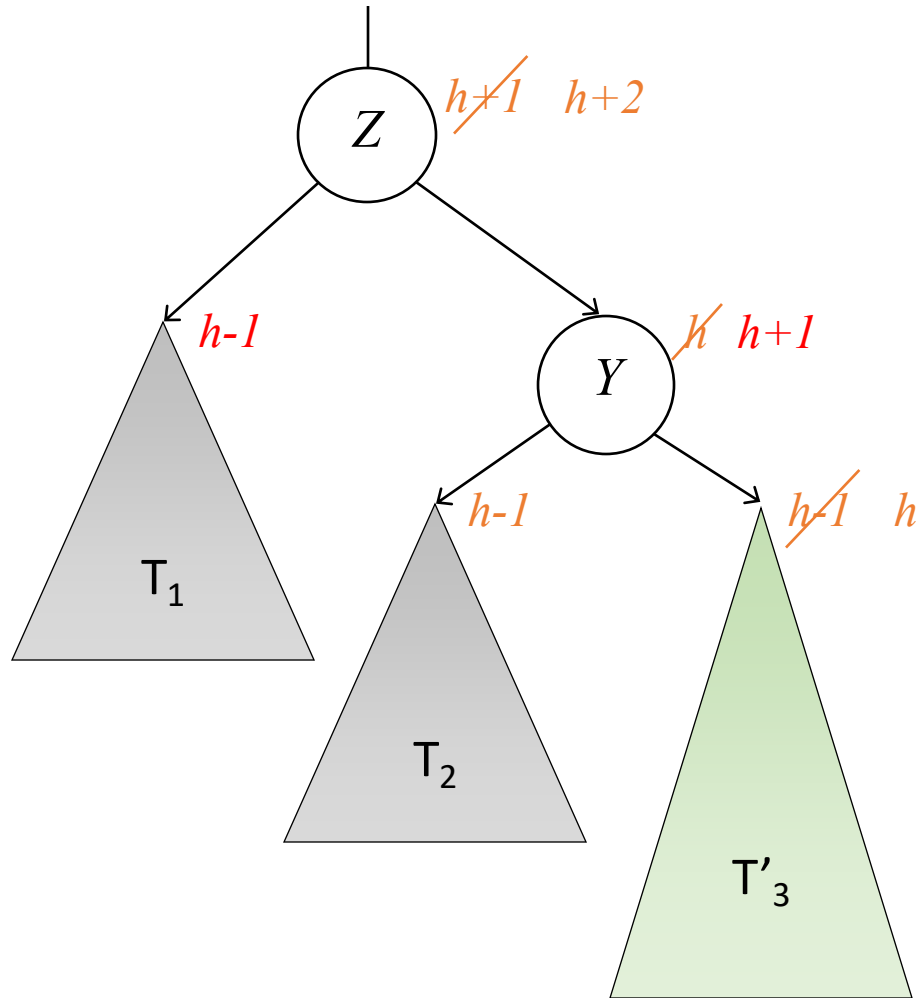
AVL Insert – Outside Case

After the Insert operation



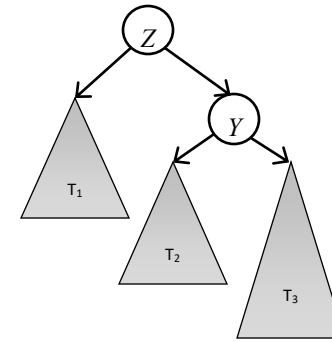
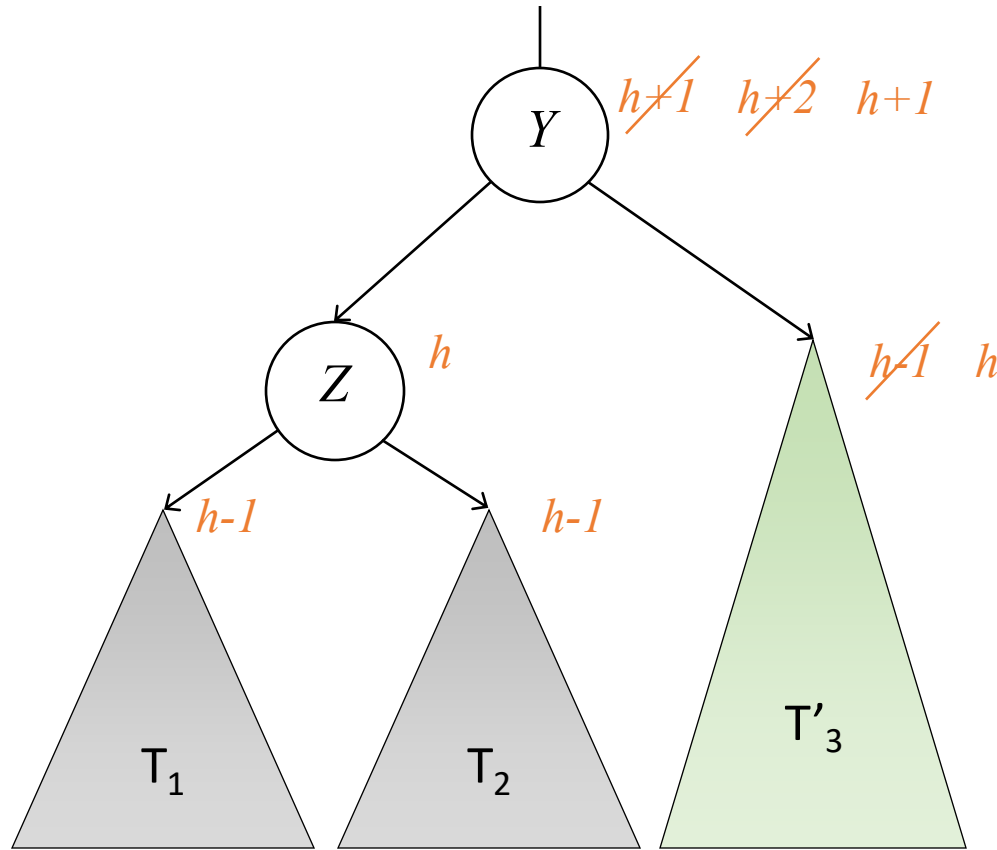
AVL Insert – Outside Case

Balancing

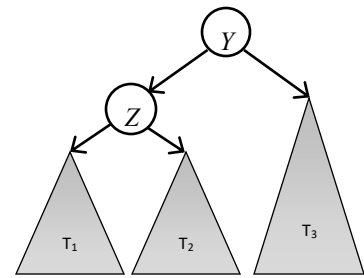


AVL Insert – Outside Case

Balancing

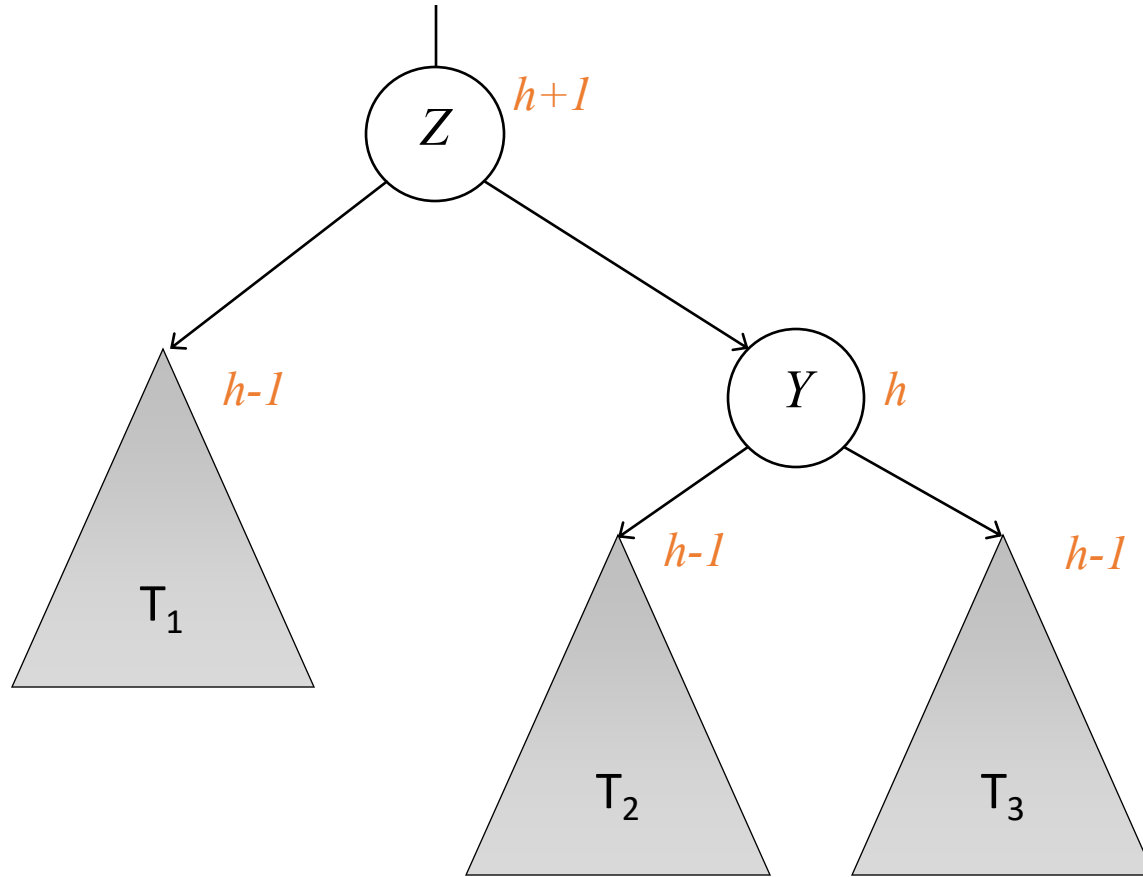


L rotation

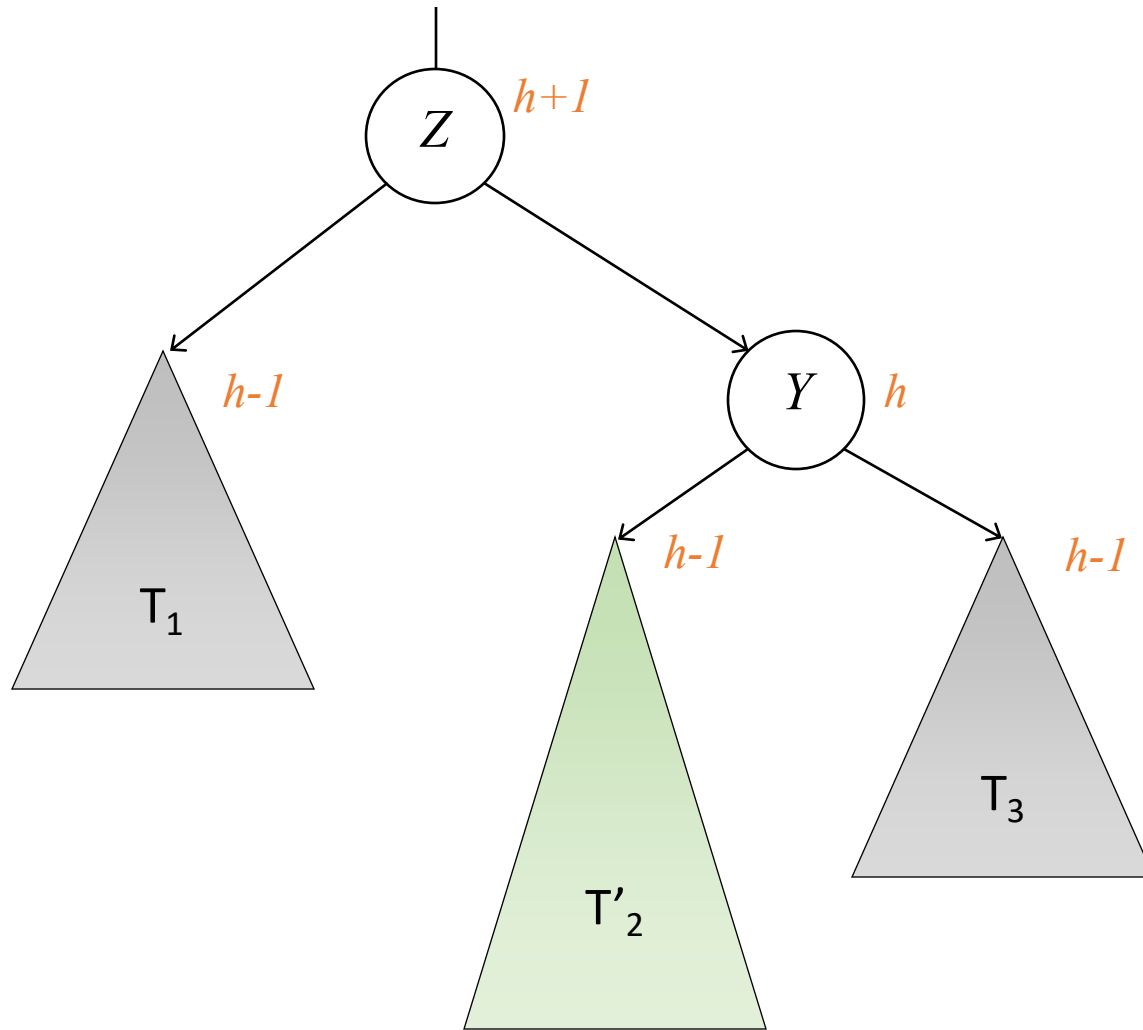


AVL Insert – Inside Case

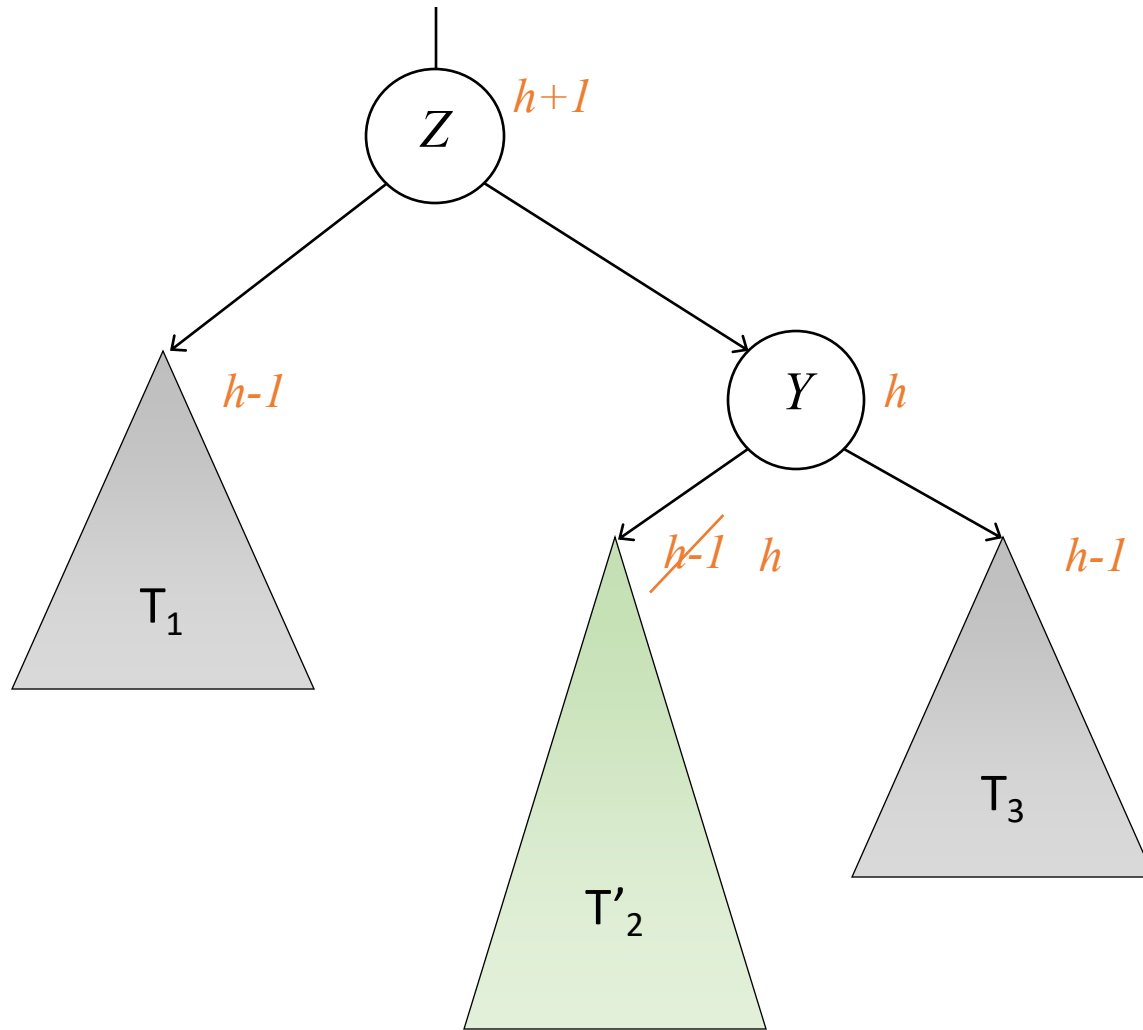
AVL Insert – Inside Case



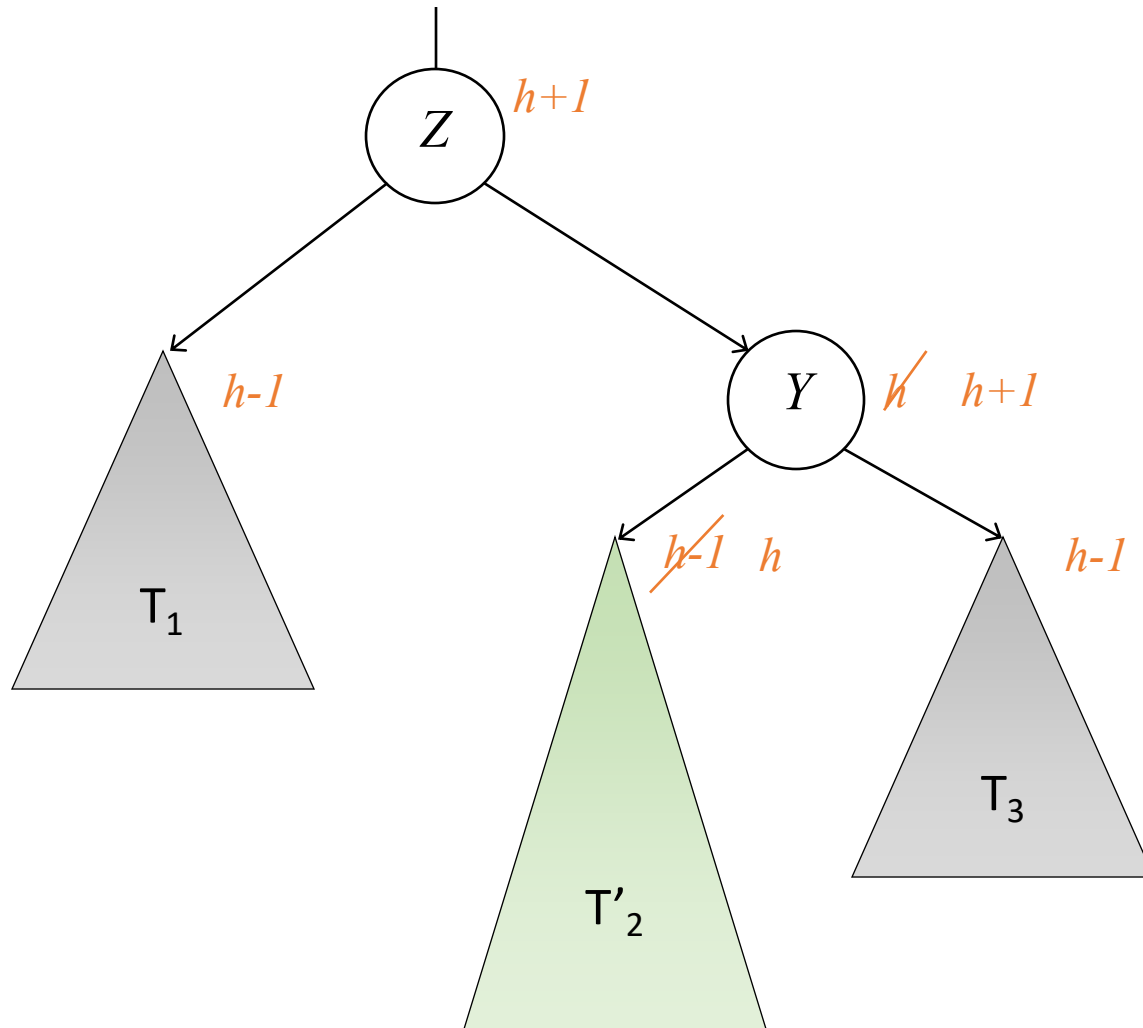
AVL Insert – Inside Case



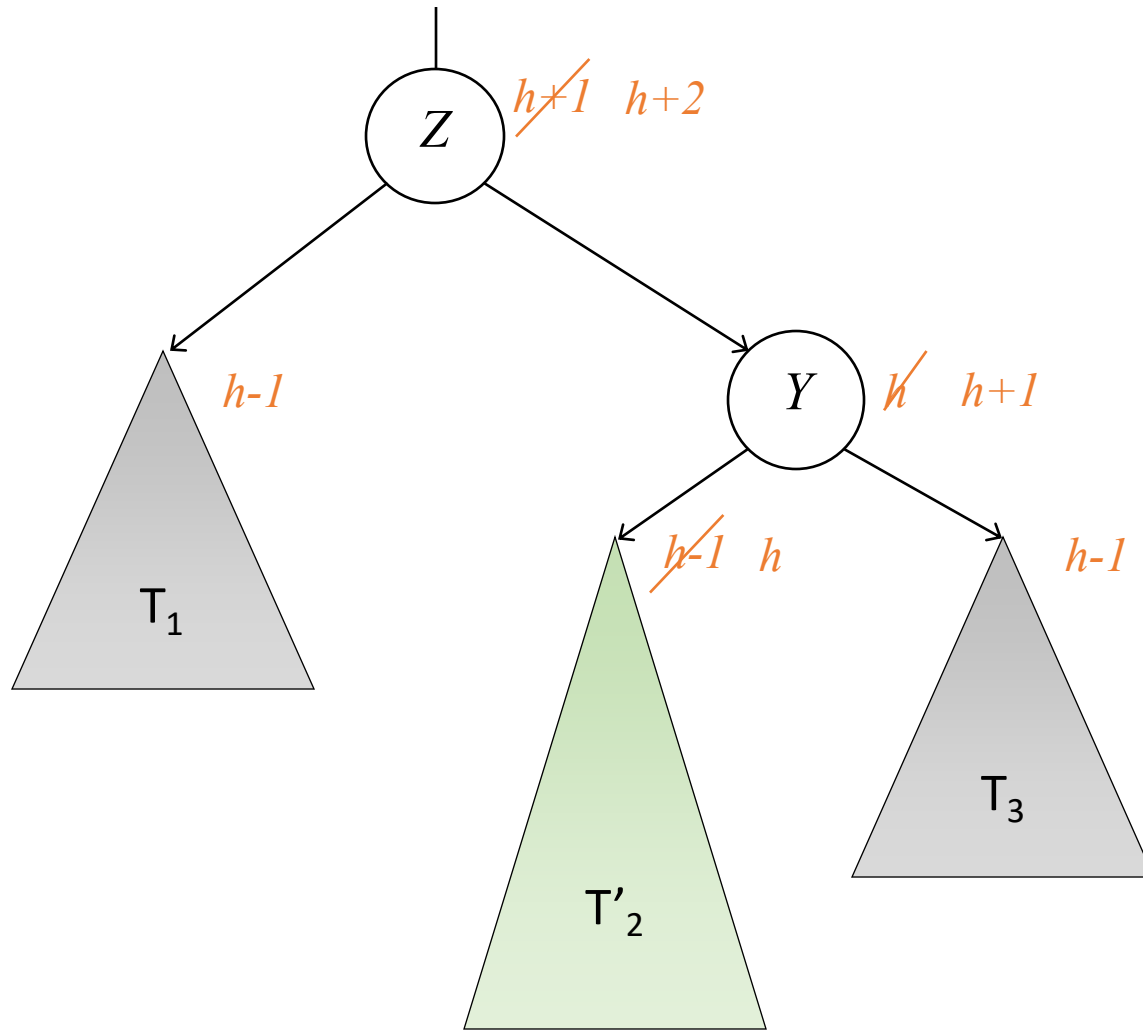
AVL Insert – Inside Case



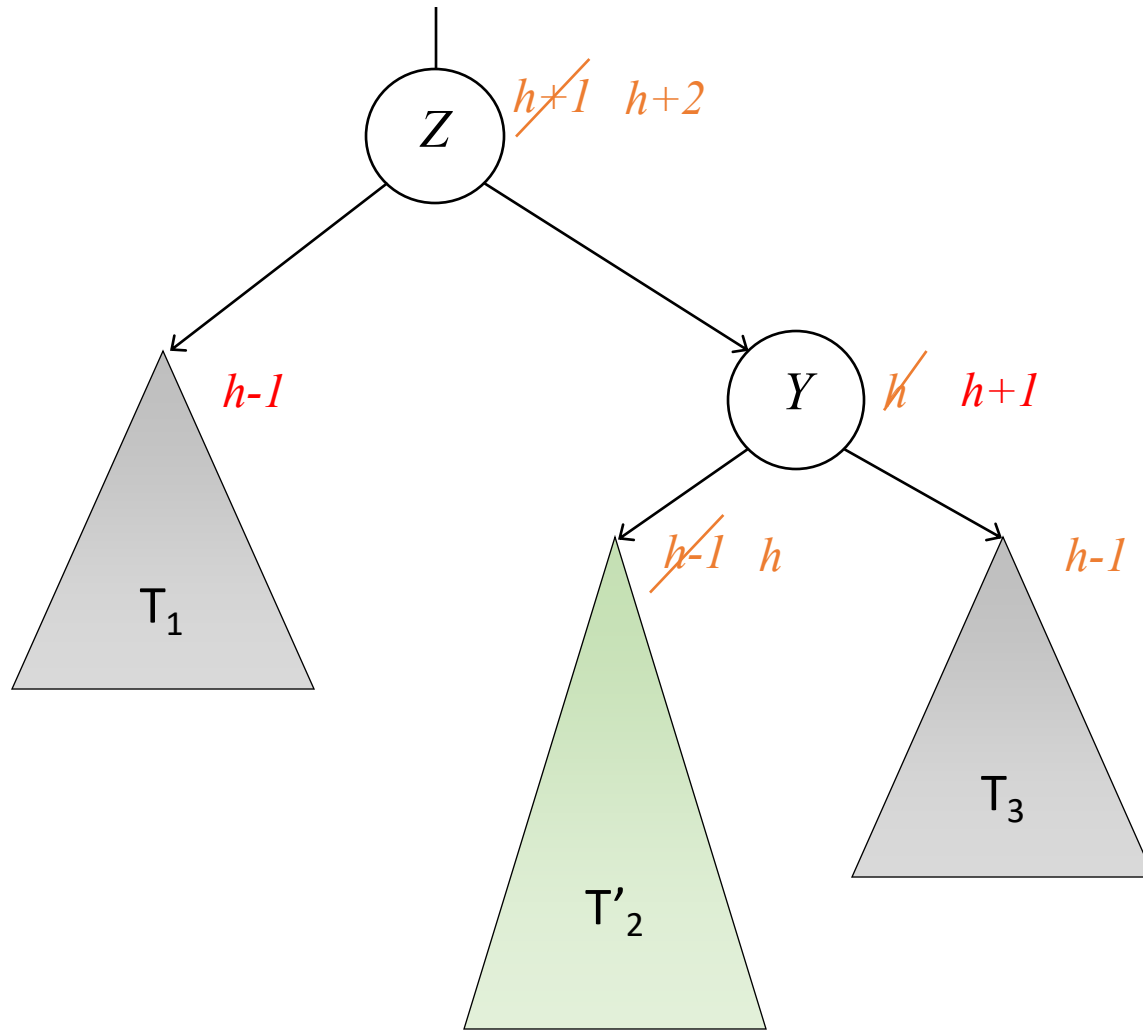
AVL Insert – Inside Case



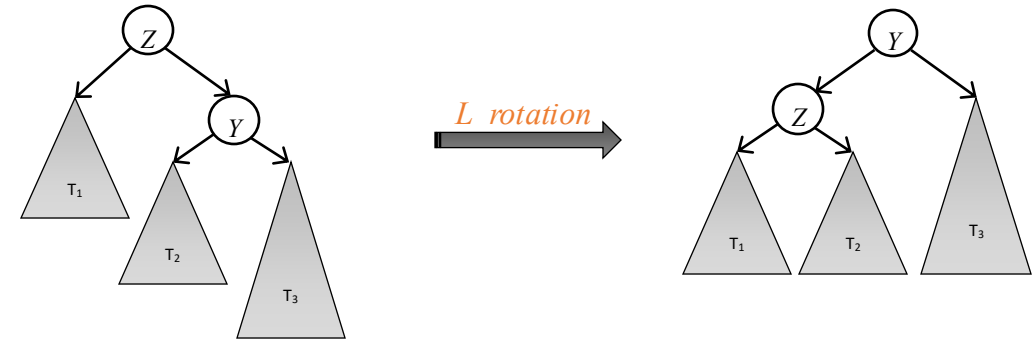
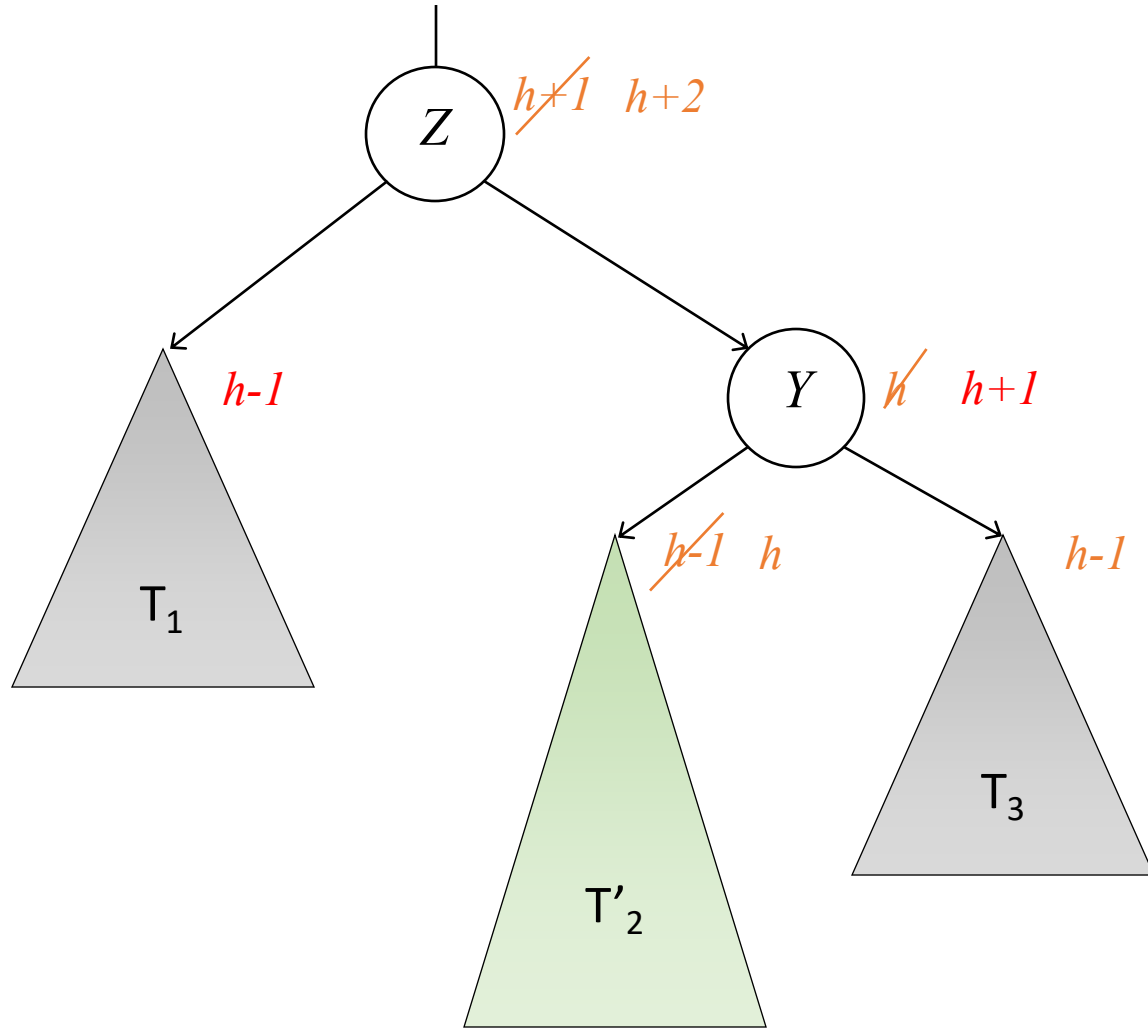
AVL Insert – Inside Case



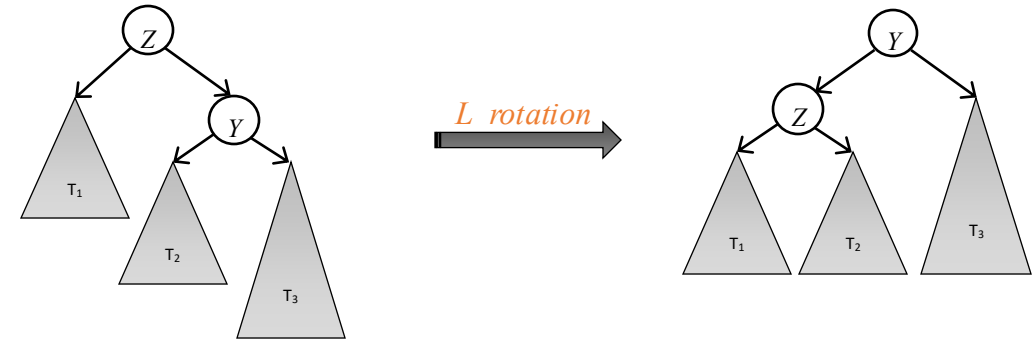
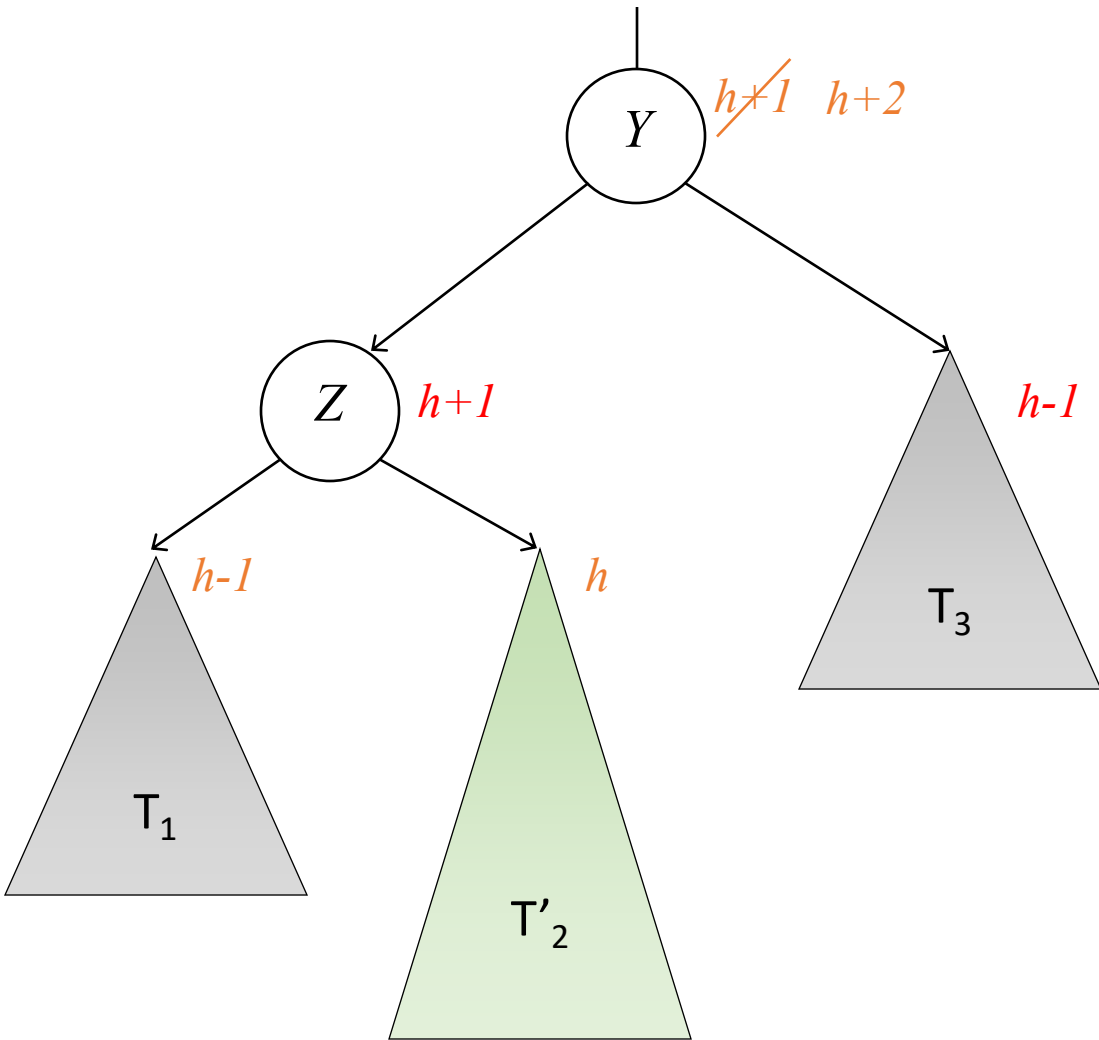
AVL Insert – Inside Case



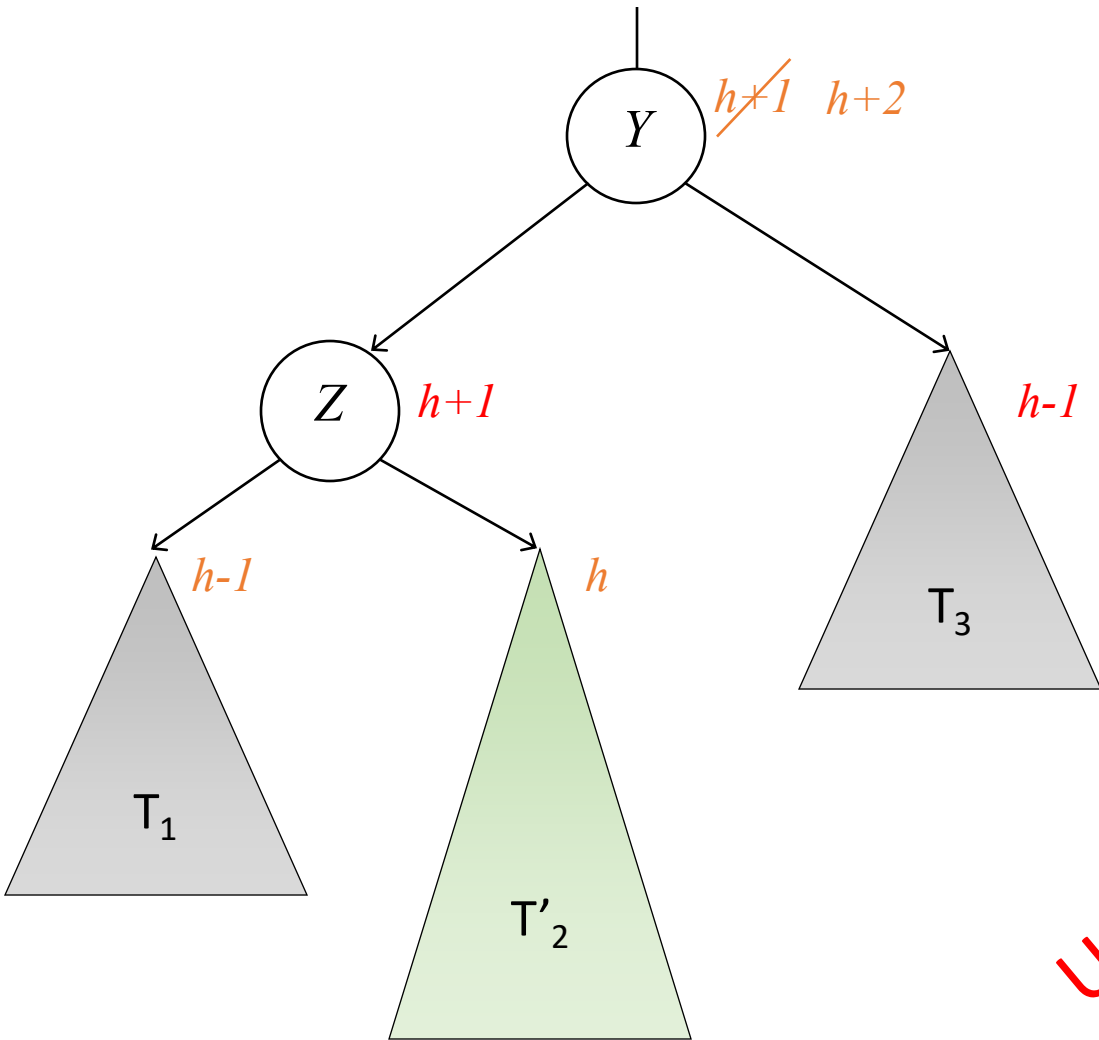
AVL Insert – Inside Case



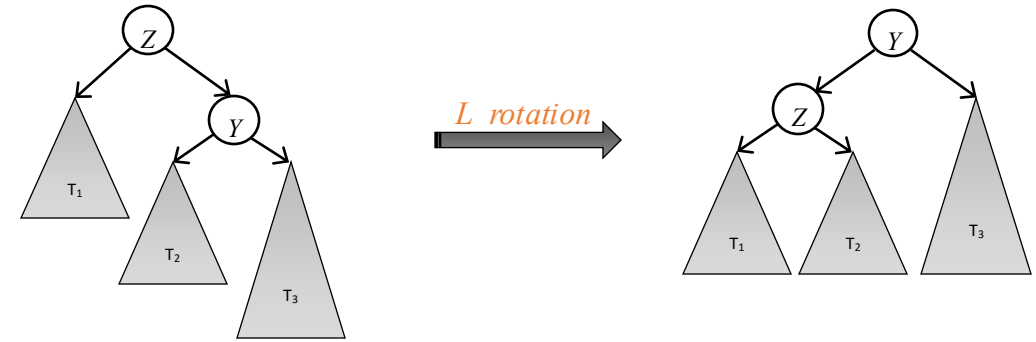
AVL Insert – Inside Case



AVL Insert – Inside Case

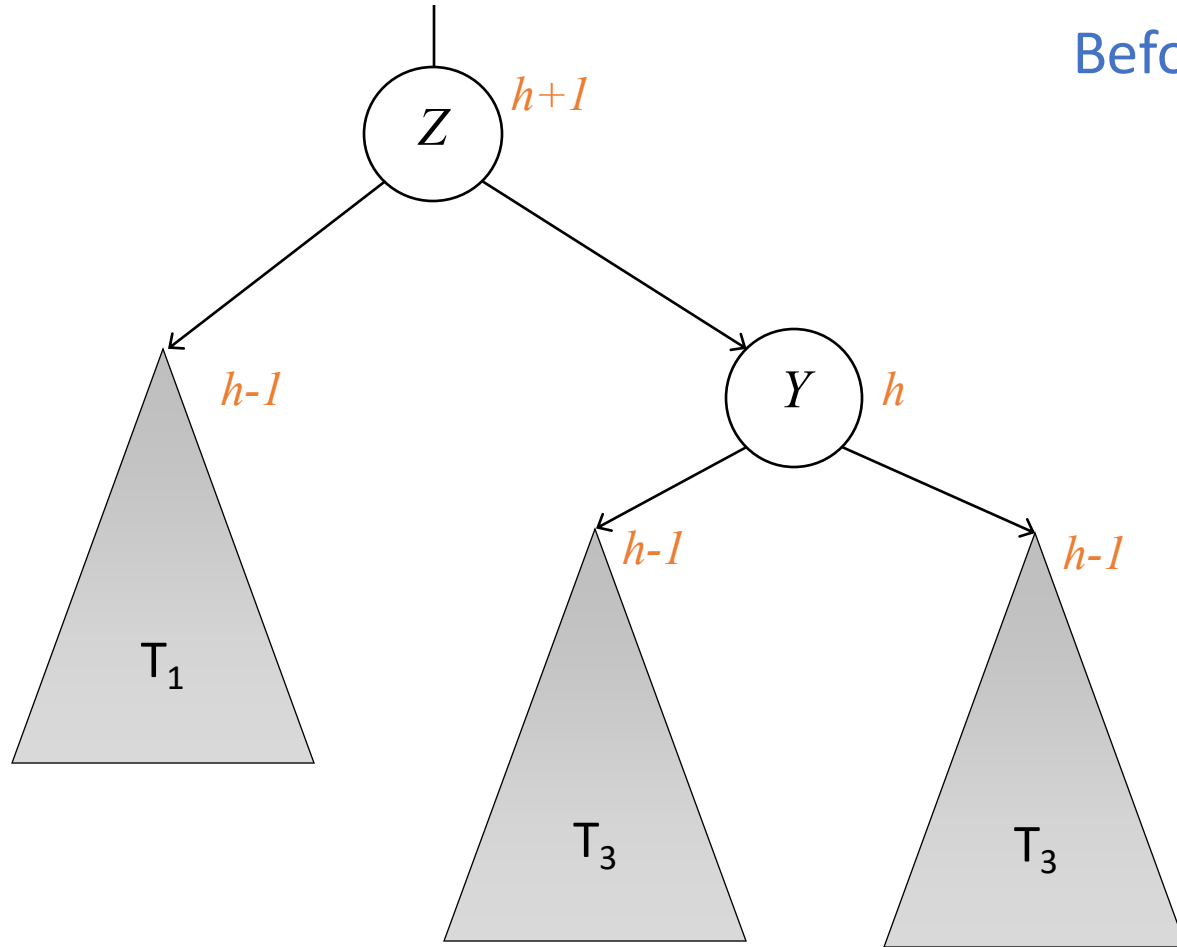


Unbalanced!



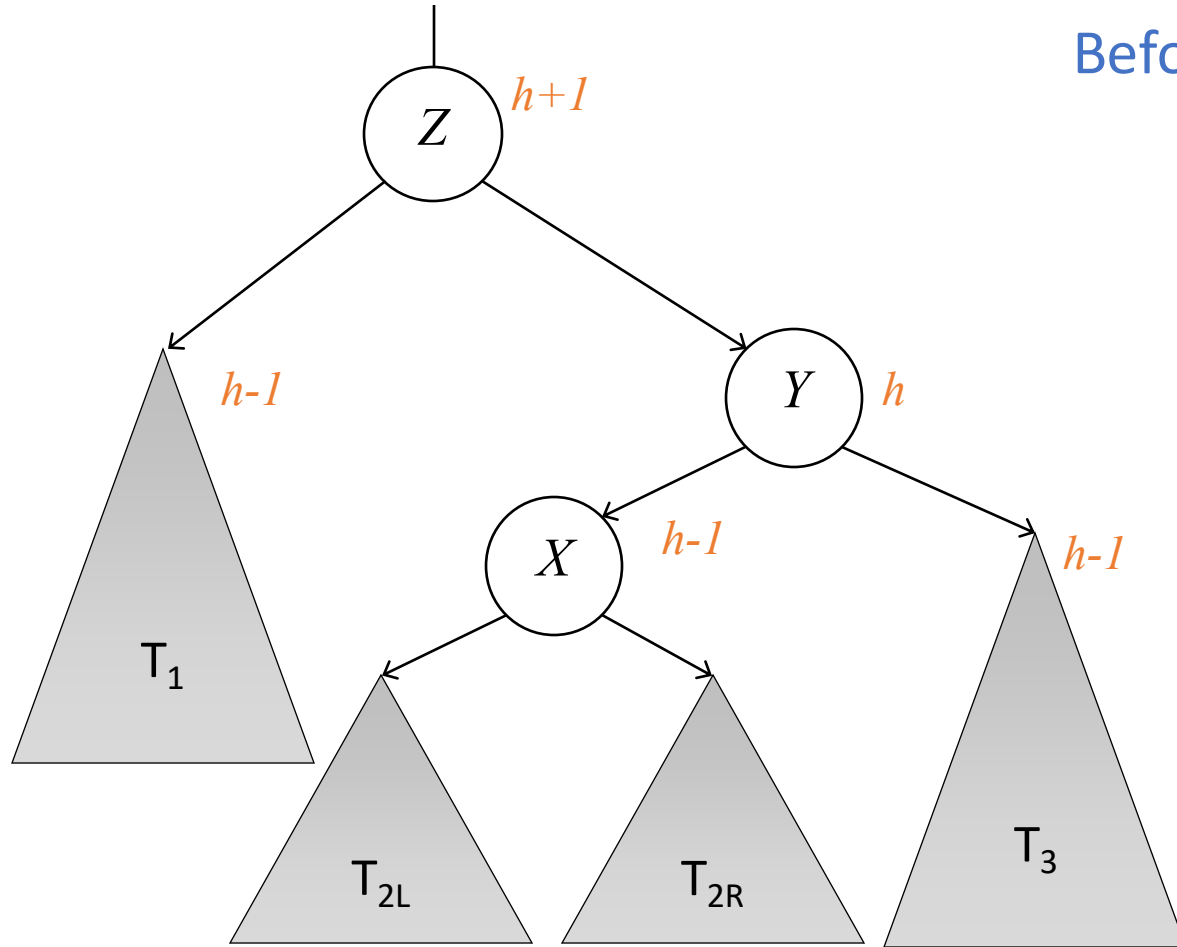
AVL Insert – Inside Case

Before the Insert operation



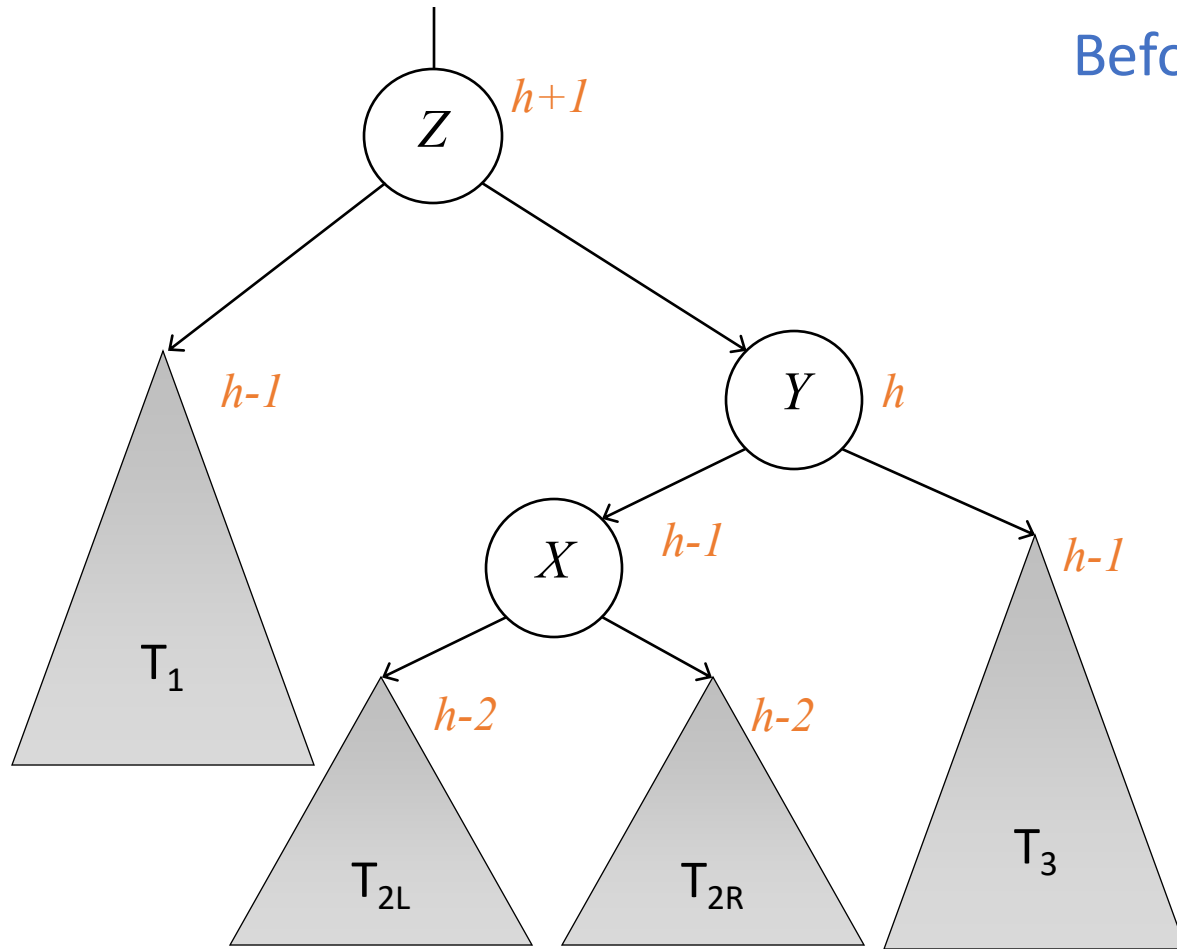
AVL Insert – Inside Case

Before the Insert operation



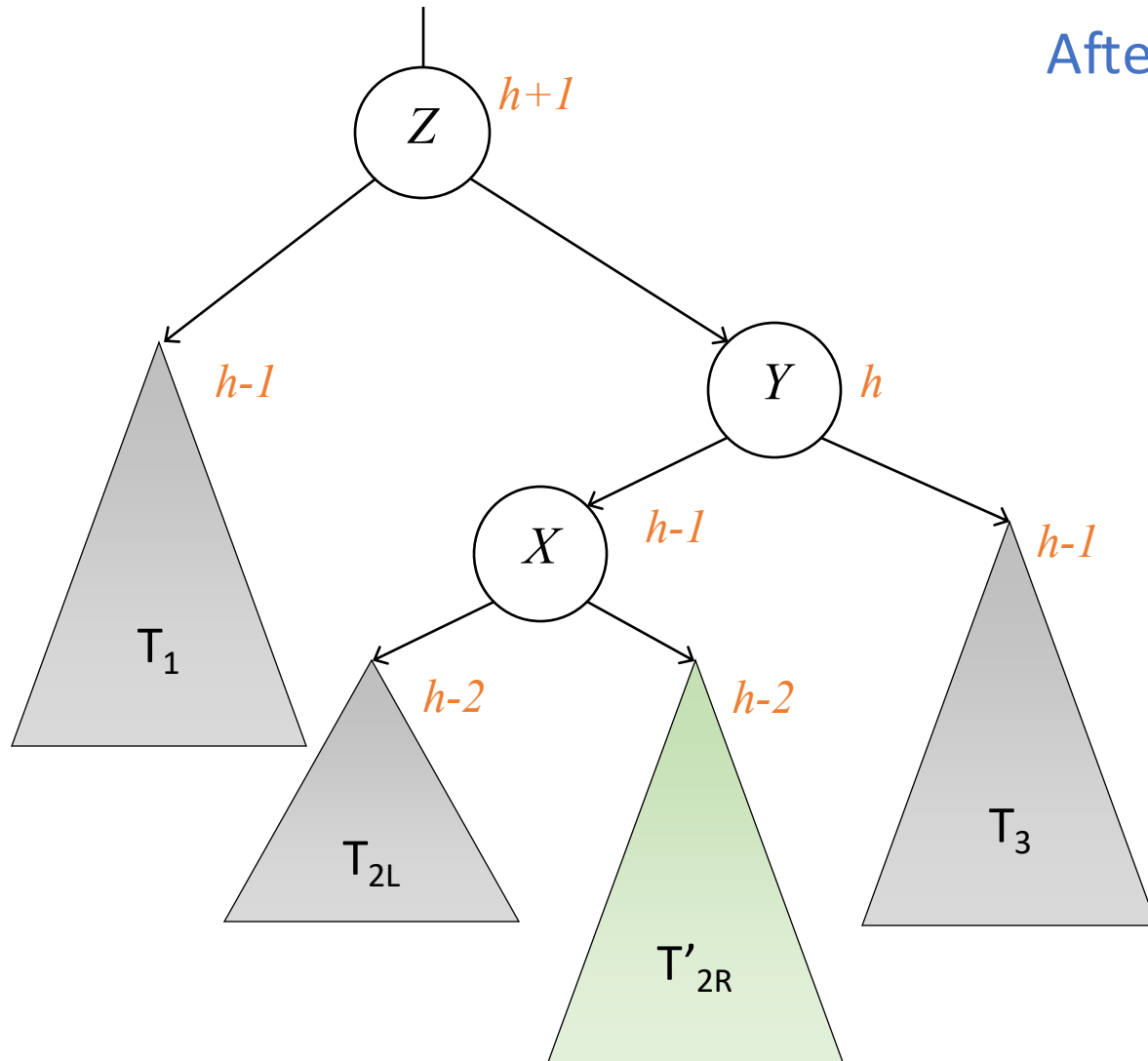
AVL Insert – Inside Case

Before the Insert operation



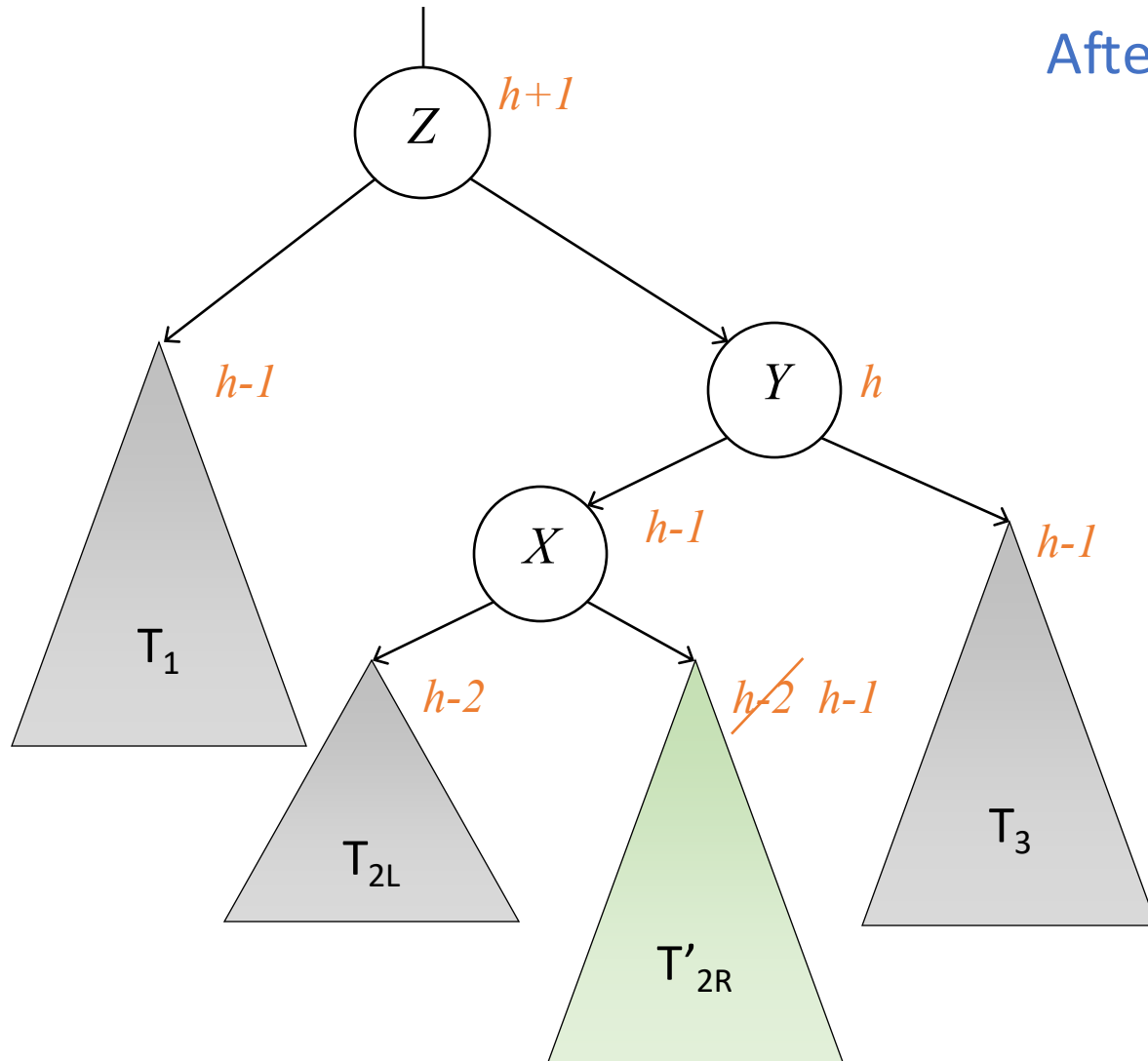
AVL Insert – Inside Case

After the Insert operation



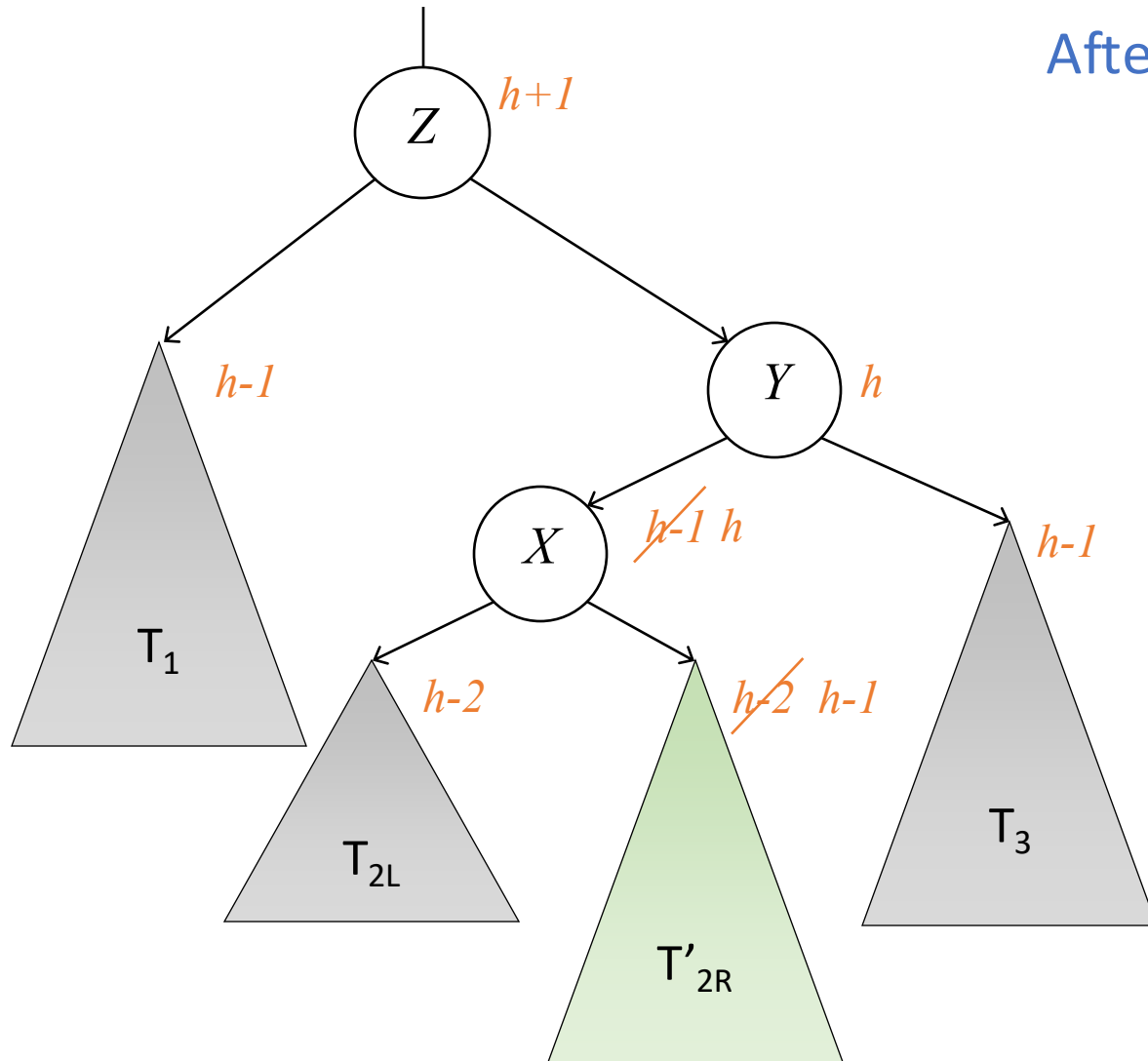
AVL Insert – Inside Case

After the Insert operation



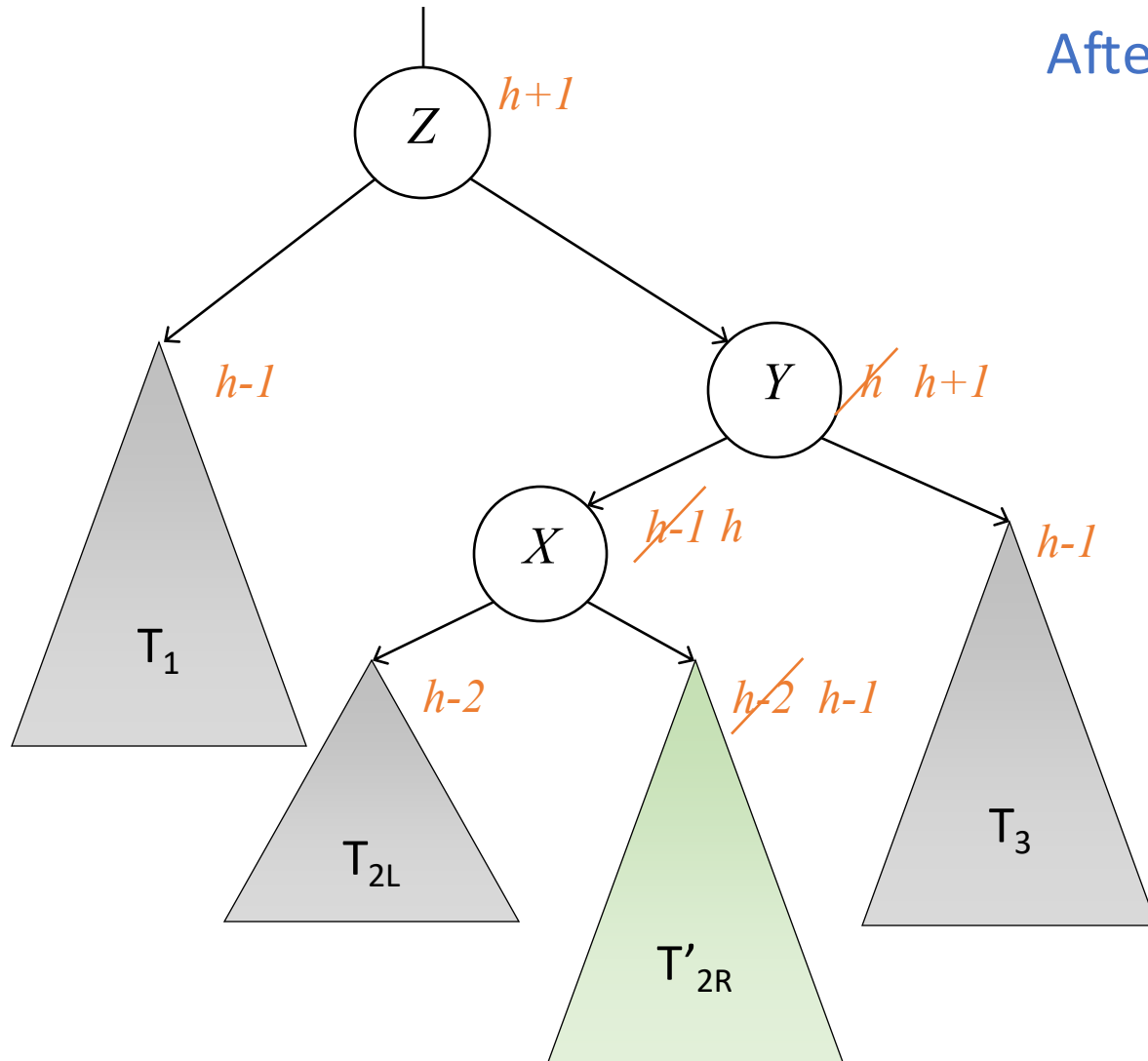
AVL Insert – Inside Case

After the Insert operation



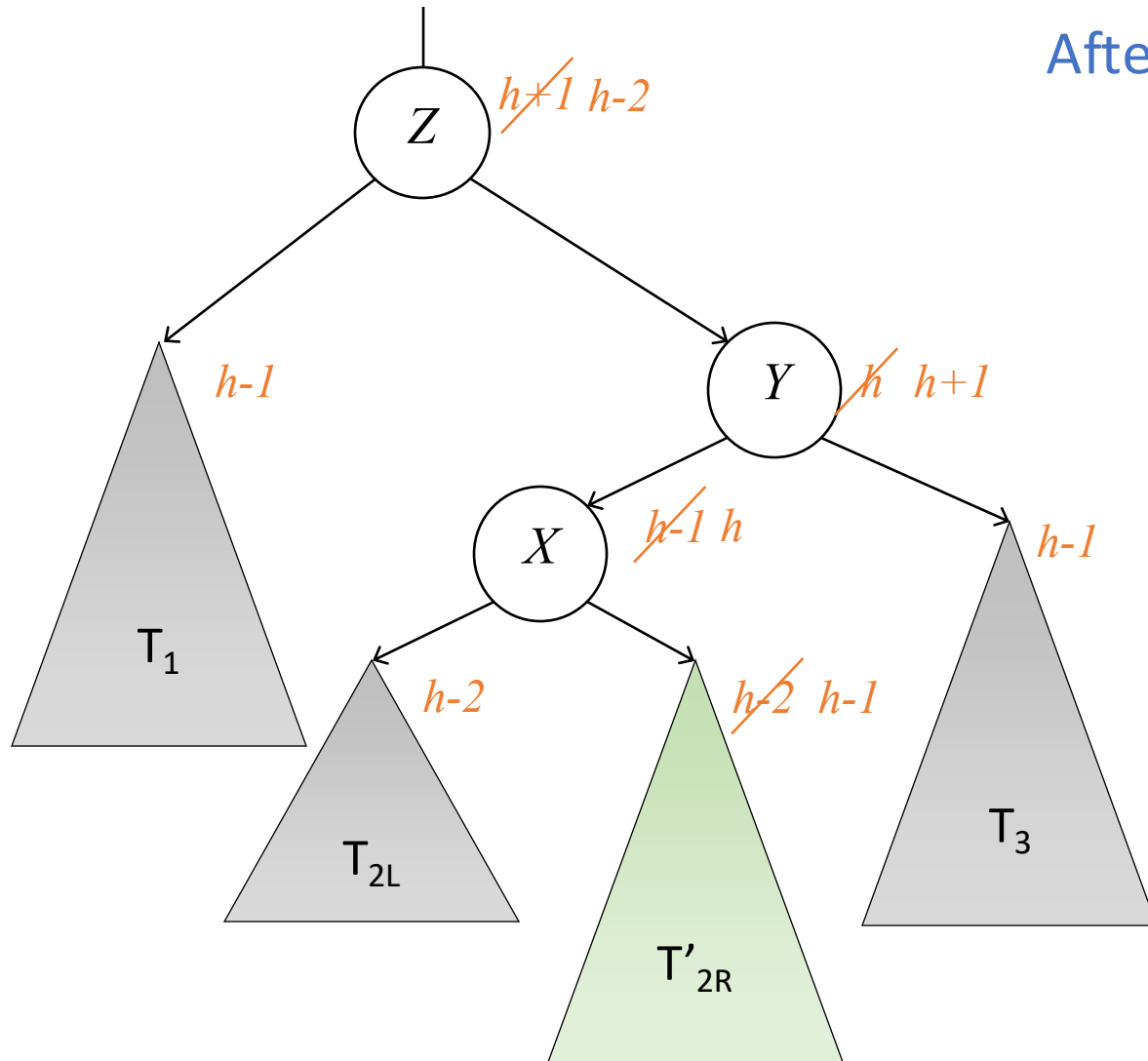
AVL Insert – Inside Case

After the Insert operation



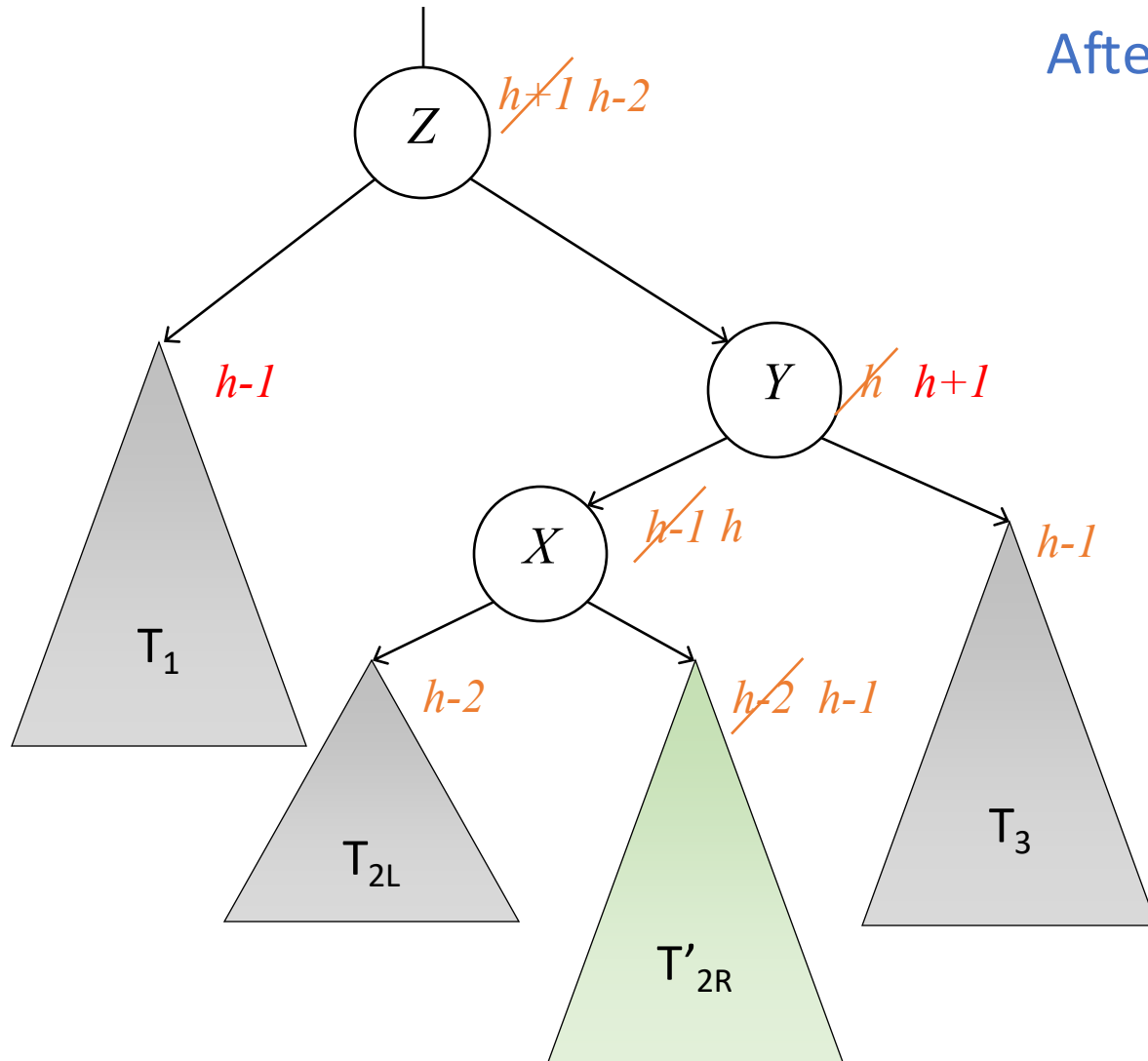
AVL Insert – Inside Case

After the Insert operation

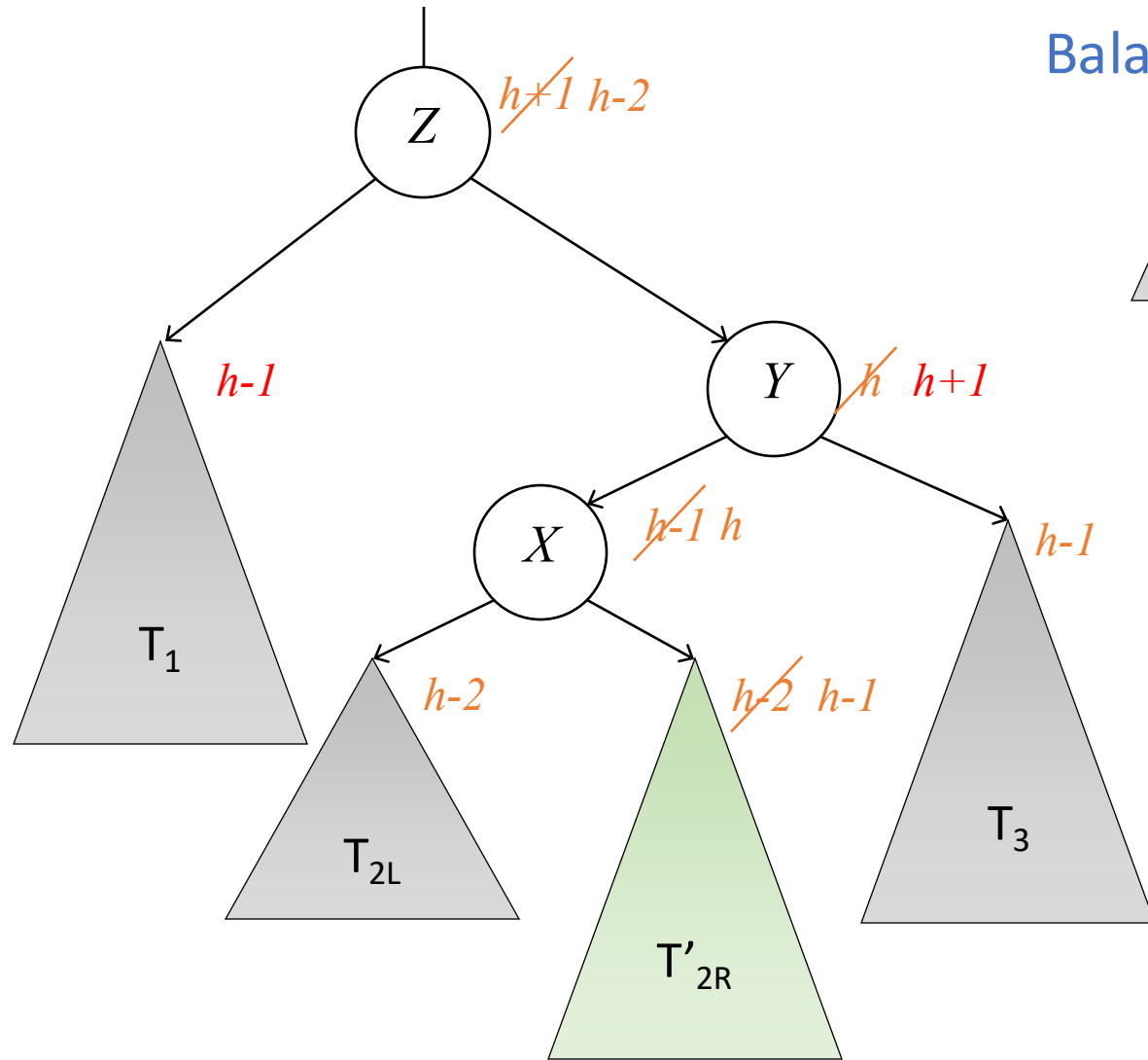


AVL Insert – Inside Case

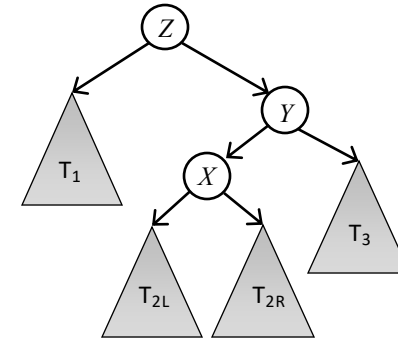
After the Insert operation



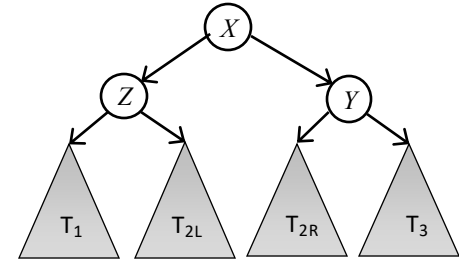
AVL Insert – Inside Case



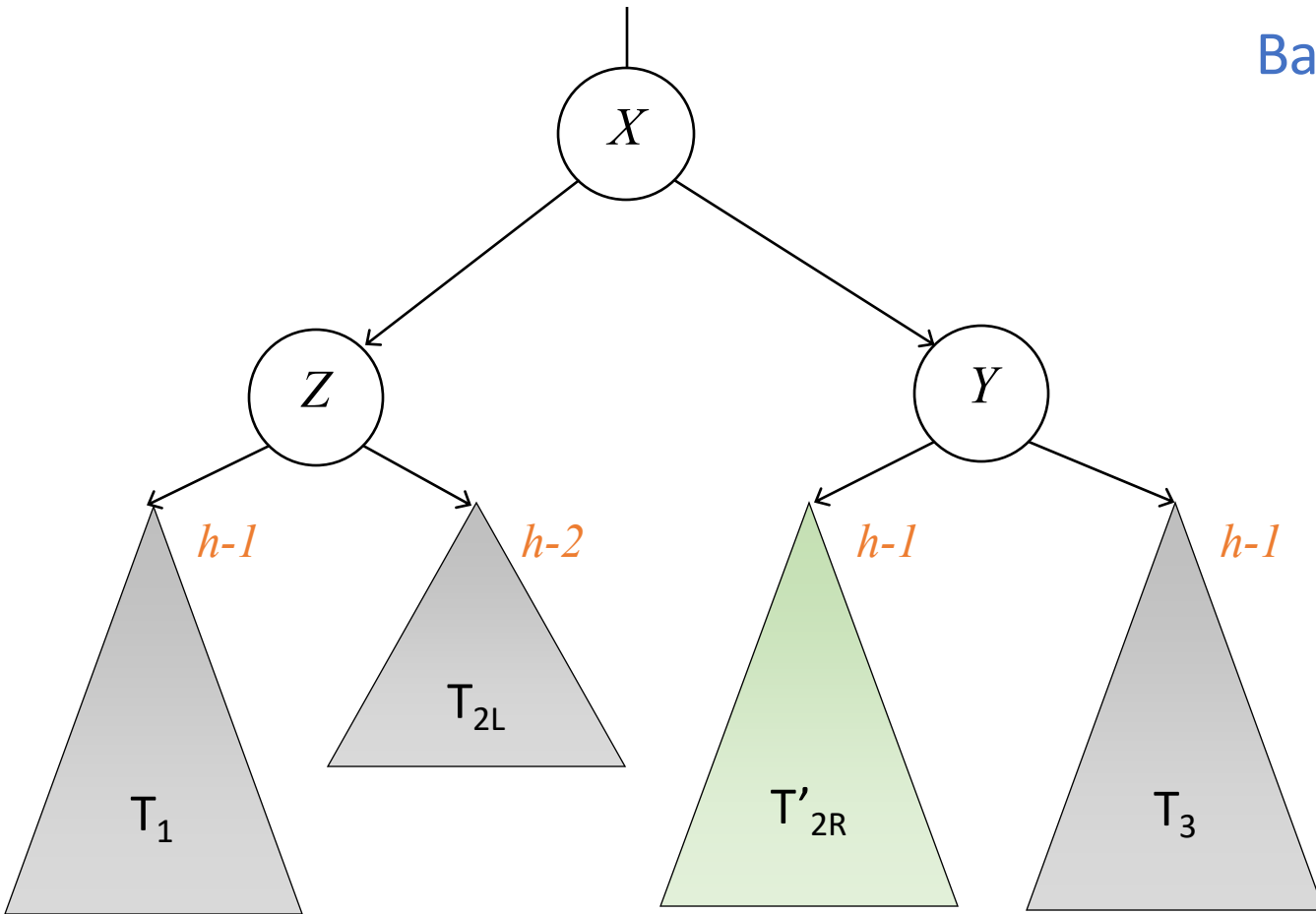
Balancing



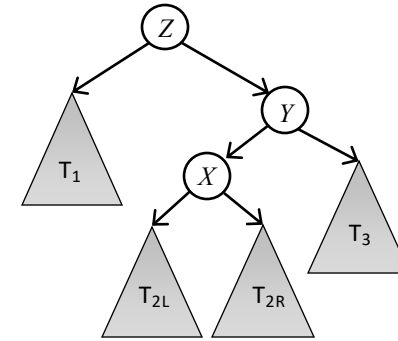
RL rotation



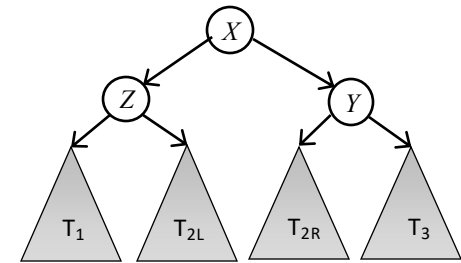
AVL Insert – Inside Case



Balancing

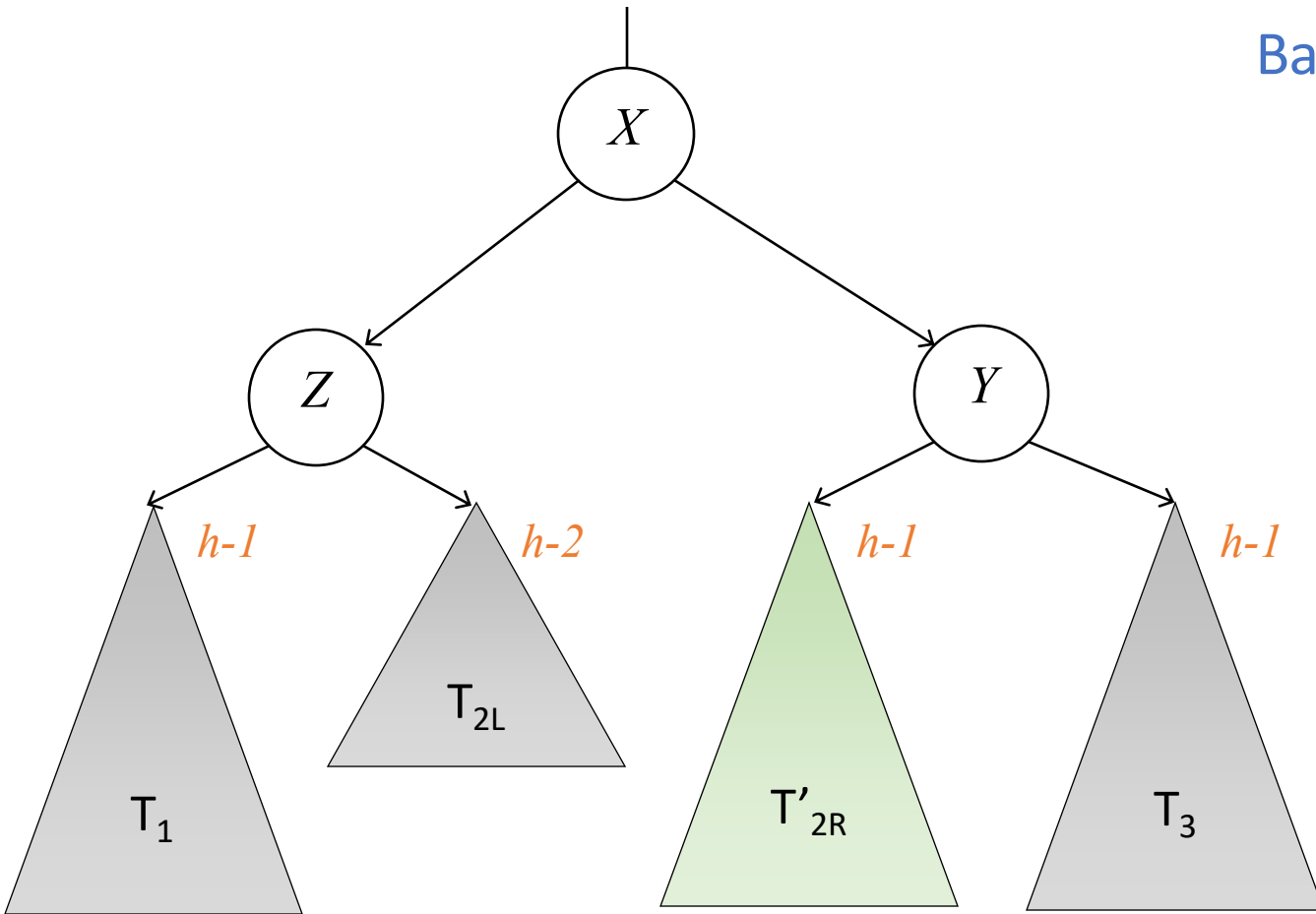


RL rotation



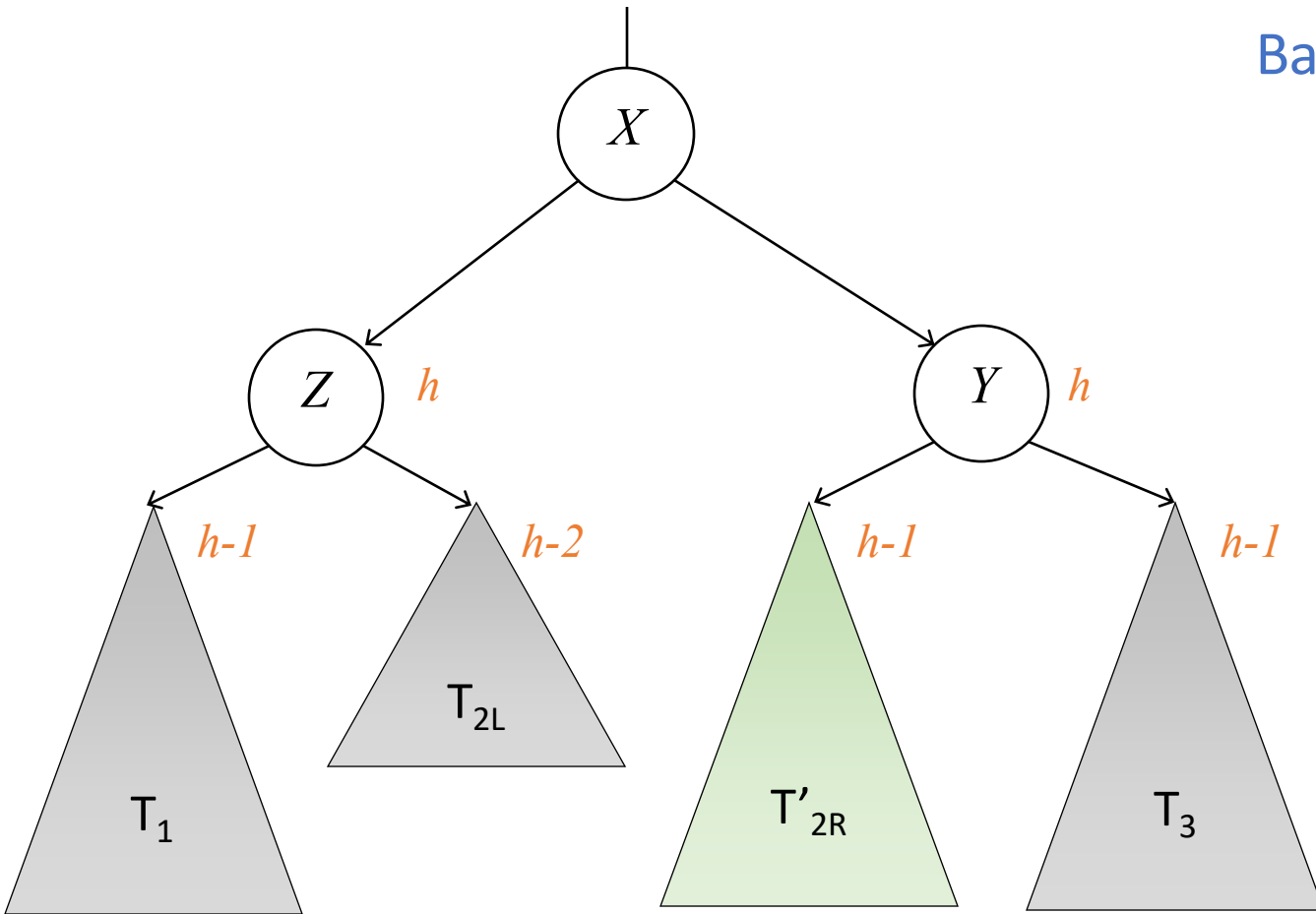
AVL Insert – Inside Case

Balancing



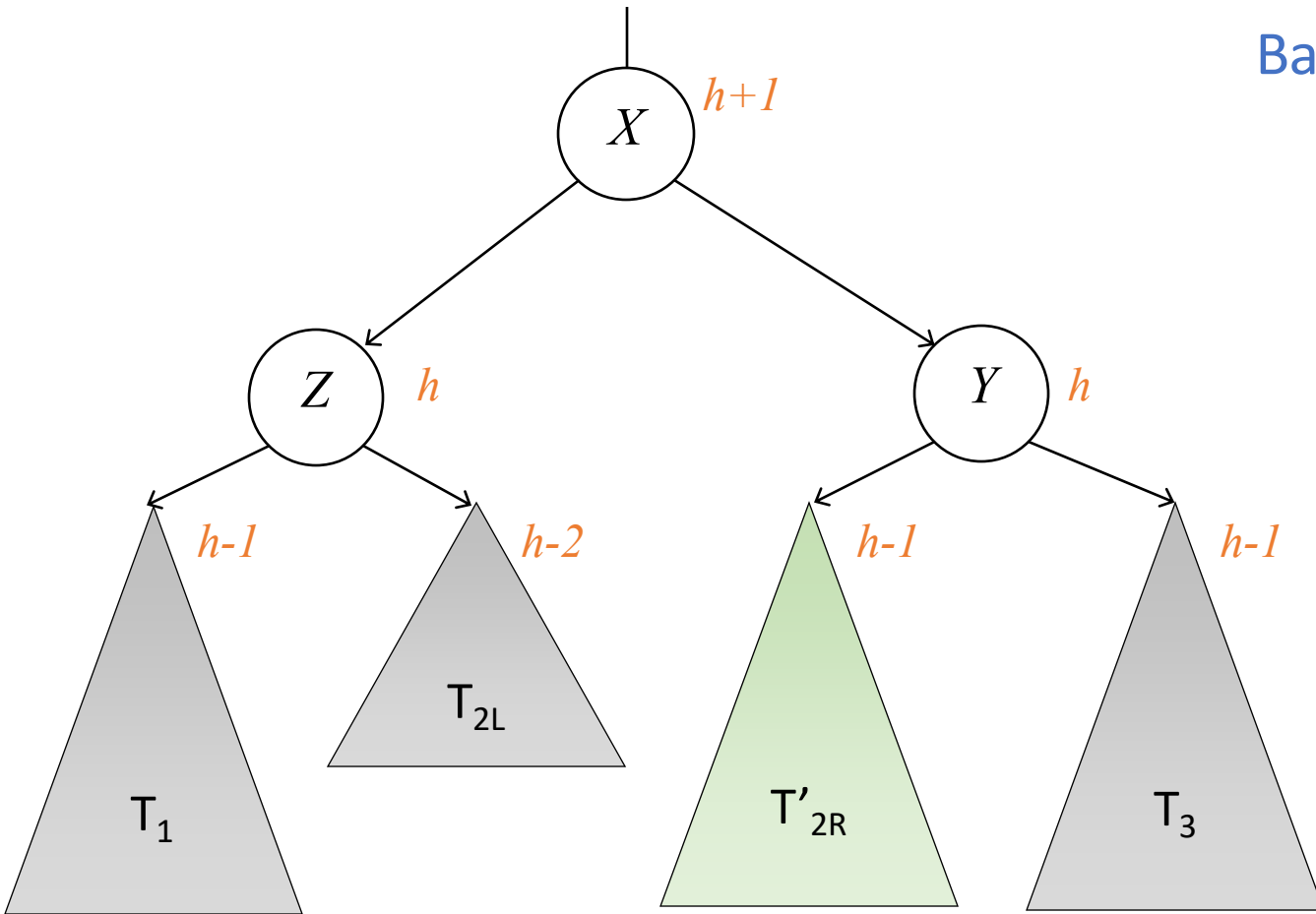
AVL Insert – Inside Case

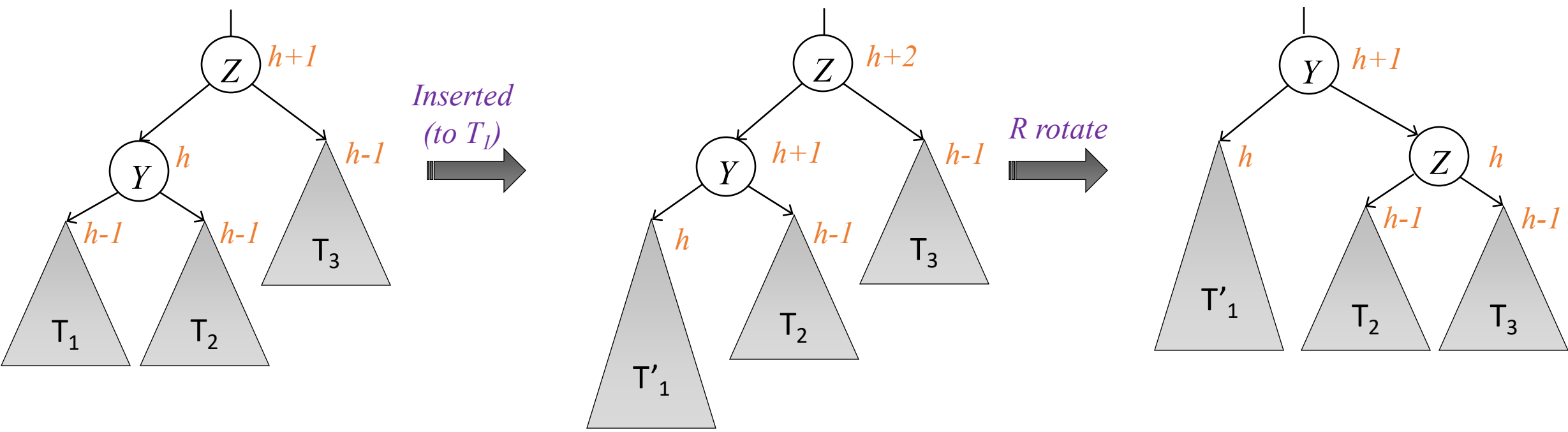
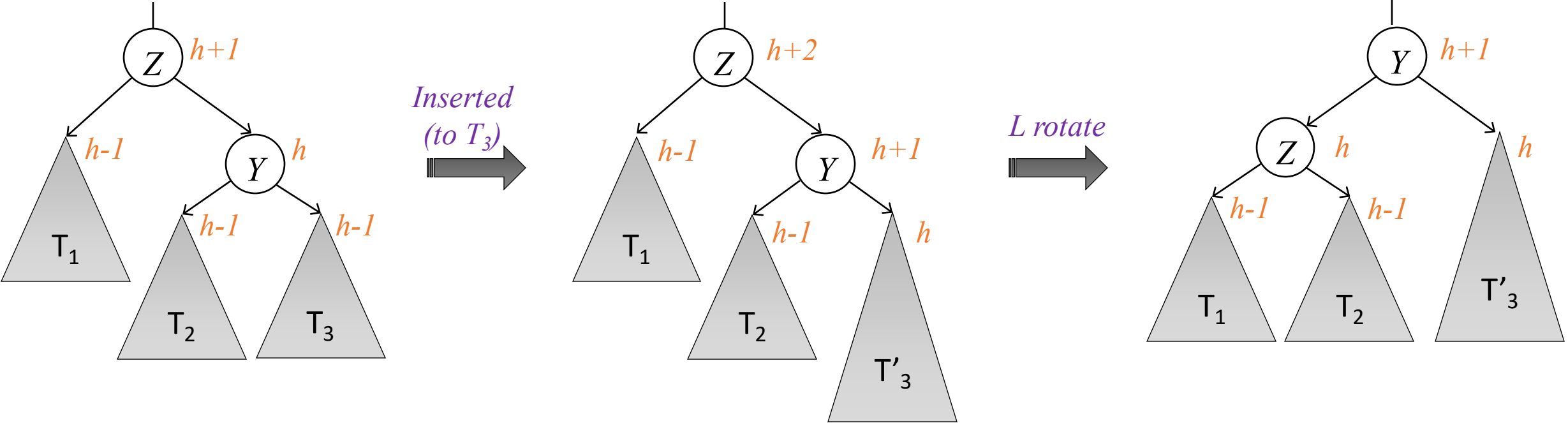
Balancing

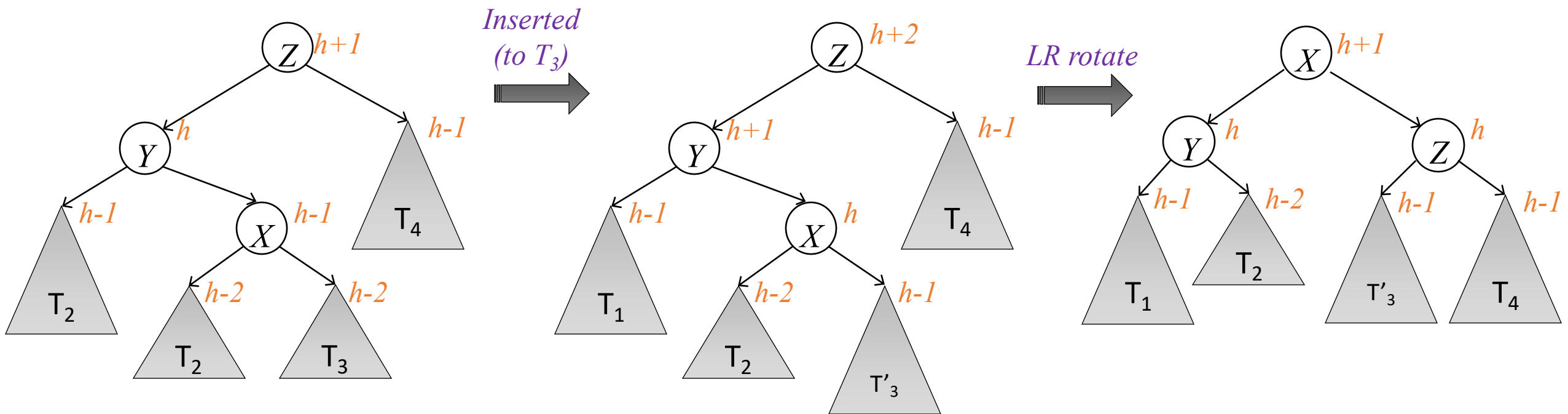
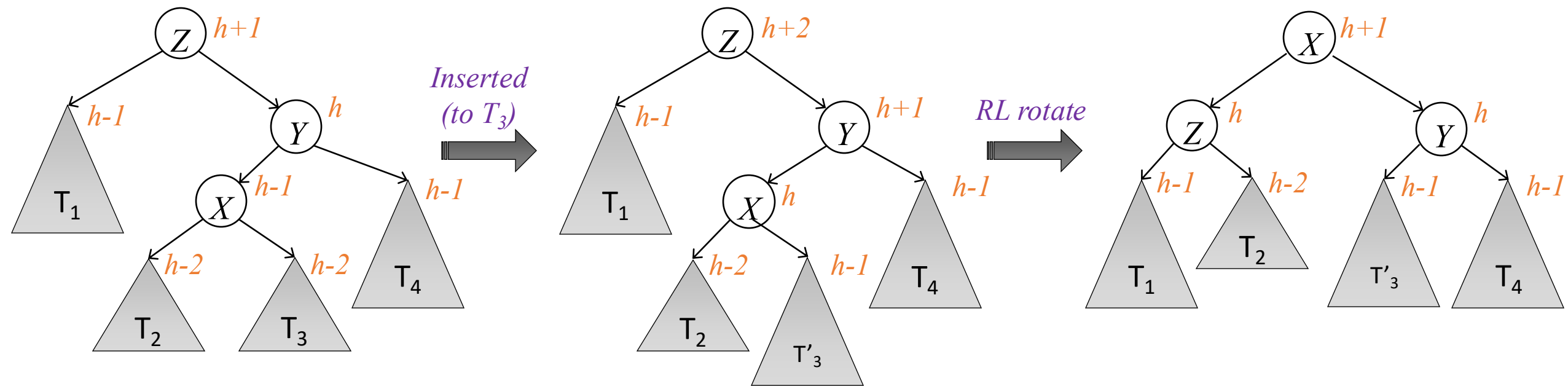


AVL Insert – Inside Case

Balancing







Data Structures for *Map ADT*

	Find	Insert	Delete
Unsorted Array	$\theta(n)$	$\theta(n)$	$\theta(n)$
Unsorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Sorted Array	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
Sorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Binary Search Tree	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$
AVL Tree			

Data Structures for *Map ADT*

	Find	Insert	Delete
Unsorted Array	$\theta(n)$	$\theta(n)$	$\theta(n)$
Unsorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Sorted Array	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
Sorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Binary Search Tree	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$
AVL Tree	$\theta(\log(n))$		

Data Structures for *Map ADT*

	Find	Insert	Delete
Unsorted Array	$\theta(n)$	$\theta(n)$	$\theta(n)$
Unsorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Sorted Array	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
Sorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Binary Search Tree	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$
AVL Tree	$\theta(\log(n))$	$\theta(\log(n))$	

Data Structures for *Map ADT*

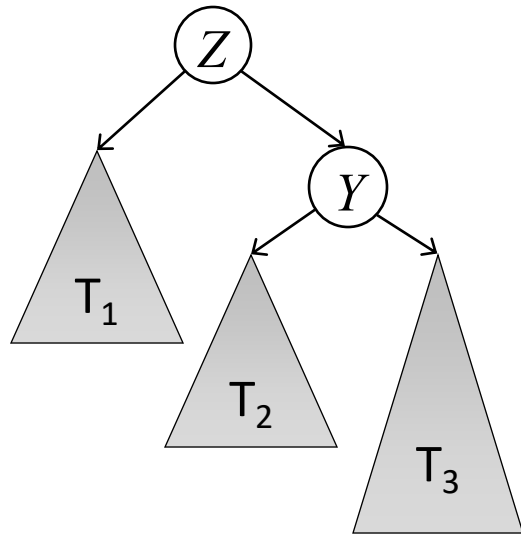
	Find	Insert	Delete
Unsorted Array	$\theta(n)$	$\theta(n)$	$\theta(n)$
Unsorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Sorted Array	$\theta(\log(n))$	$\theta(n)$	$\theta(n)$
Sorted Linked List	$\theta(n)$	$\theta(n)$	$\theta(n)$
Binary Search Tree	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$	$\theta(h), \theta(n)$
AVL Tree	$\theta(\log(n))$	$\theta(\log(n))$	$\theta(\log(n))$

AVL Insert

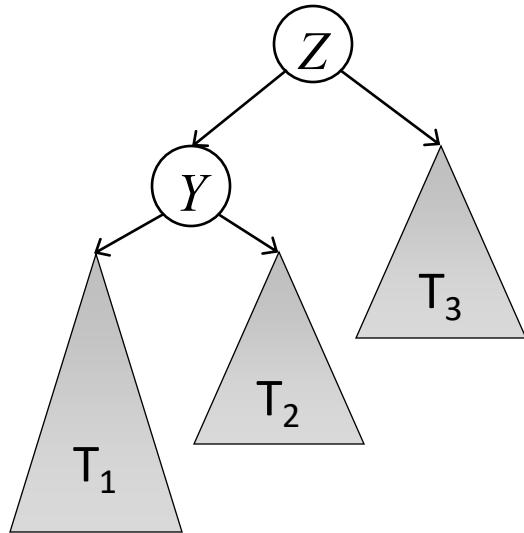
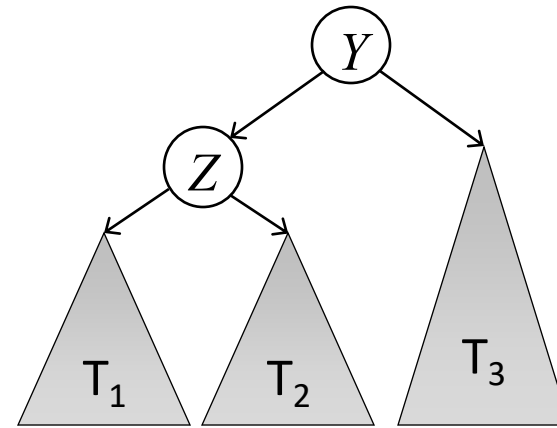
Insert k into an *AVL tree*:

- Apply the insert algorithm of k as if the tree is a regular *Binary Search Tree*. As a result, the balancing property could be violated
- Walk from the new node (with k) upwards on the insertion path.
- Let z be the first node in this walk that is not balanced (The difference between z 's two children is 2)
- Let y be z 's higher child (k was inserted into y 's subtree)
- Determine if you are in the outside-case or in the inside-case
 - If you are in the outside-case do the corresponding single rotation
 - If you are in the inside-case do the corresponding double rotation

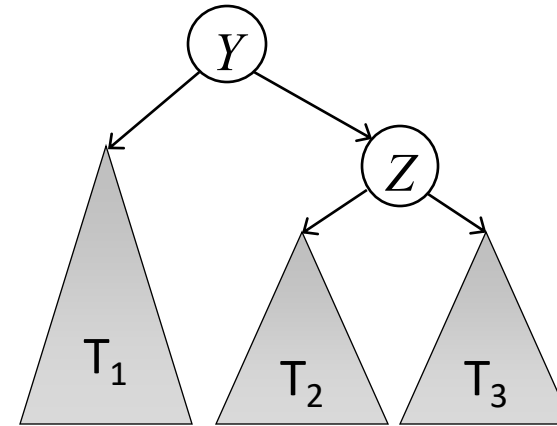
Single Rotations



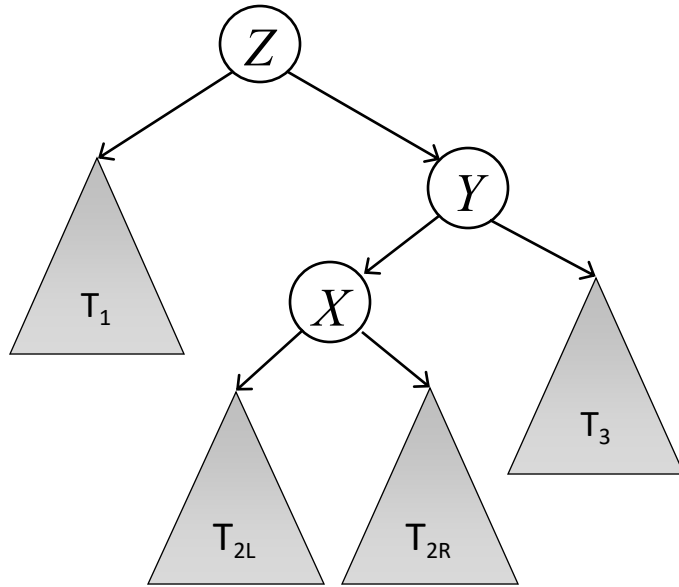
L rotation



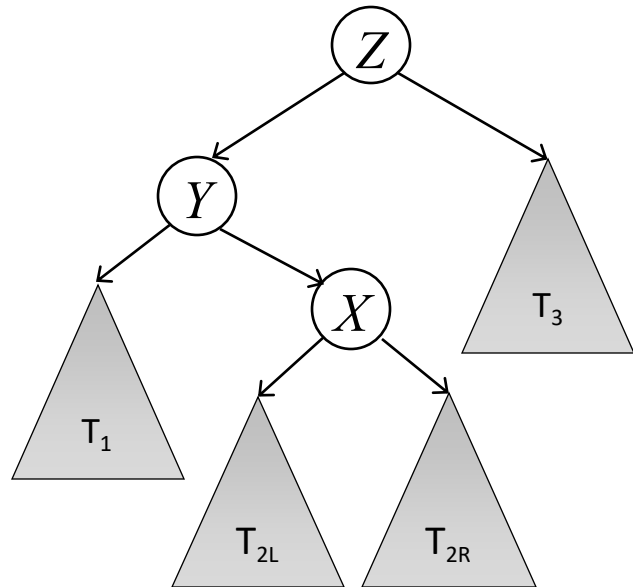
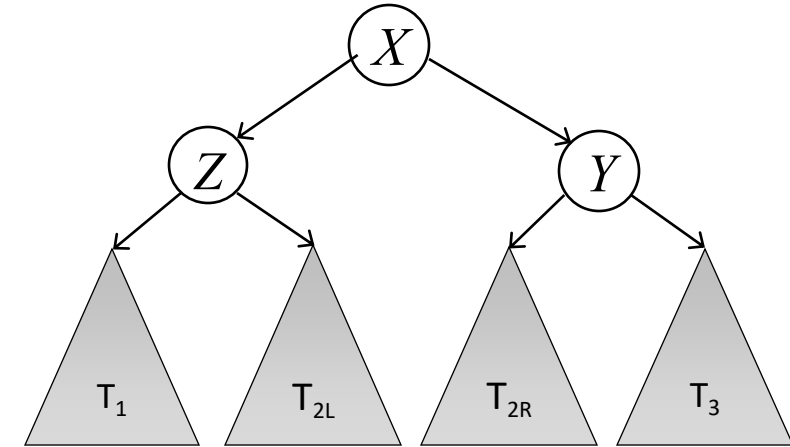
R rotation



Double Rotations



RL rotation



LR rotation

